Estudiante: David Mauricio Pérez Cordero | **Documento:** xxxxxxxx2

Plataforma de Inscripción Académica (PIA) - Universidad de la Salle

Propuesta Integral de Arquitectura de Software - Parcial 2

1. Resumen Ejecutivo

La **Universidad de la Salle** cuenta con una comunidad académica de aproximadamente **24.000 estudiantes, 1.100 docentes y 5 facultades**. Actualmente, el proceso de matrícula de asignaturas se realiza a través de un sistema académico desarrollado hace más de 12 años, el cual colapsa periódicamente debido a lentitud extrema, caídas de disponibilidad, inconsistencias de cupos, inscripciones duplicadas y alta dependencia síncrona.

El presente repositorio contiene la **Propuesta de Arquitectura de Software para la nueva Plataforma de Inscripción Académica (PIA)**. La solución ha sido diseñada bajo los más rigurosos estándares de la ingeniería de software para operar como una autoridad moderna de matrículas capaz de resolver los problemas históricos de la institución y operar de forma fluida bajo condiciones extremas de demanda.

Logros Clave del Diseño Arquitectónico:

1. **Garantía Matemática de Cero Sobrecupos (`C04`):** Eliminación absoluta de sobrecupos en grupos académicos mediante sentencias atómicas condicionales con bloqueo ultracorto a nivel de fila (*RowExclusiveLock*) en PostgreSQL.
2. **Alta Concurrencia e Idempotencia (Escenarios A y B / `C01`, `C02`):** Capacidad probada para absorber 1.200 usuarios concurrentes nominales y hasta 1.800 en horas pico, neutralizando ráfagas de hasta 5 reintentos automáticos mediante llaves `Idempotency-Key` y sirviendo el 95% de las consultas de catálogo en menos de 150 ms (superando la meta de 2,0 segundos).
3. **Resiliencia ante Caídas del Sistema Legado (`C05` y Escenario A):** Desacoplamiento temporal de la validación de prerrequisitos mediante una réplica local pre-sincronizada a T-24h y encolamiento transaccional de eventos (*Transactional Outbox Pattern* con *Circuit Breaker*), permitiendo que la PIA opere al 100% durante interrupciones del legado de hasta 40 minutos sin perder transacciones ni violar la disponibilidad del 99,9% (`C03`).
4. **Continuidad Operativa RTO $\le$ 30 min y RPO $\le$ 5 min (`C06`, `C07`):** Clúster de alta disponibilidad en PostgreSQL con archivado continuo de segmentos WAL cada 60 segundos hacia almacenamiento seguro secundario (RPO real $\le$ 1 min, RTO real $\le$ 10 min).
5. **Máxima Eficiencia y Factibilidad Operativa (`C08`, `C10`):** Implementada 100% sobre servidores **Linux y PostgreSQL estándar**, sin costos en servicios de nube comerciales de pago y con un modelo de gobernanza ágil

(*Governance-as-Code*) perfectamente gestionable por el equipo institucional de **4 desarrolladores y 1 administrador de infraestructura**.

---

2. Mapa de Navegación del Repositorio y Entregables

La documentación se encuentra estructurada modularmente en el directorio `docs/`:

Documento	Descripción y Alcance	Vínculo Directo
Arbitraje de Requisitos	Evaluación formal de tensiones, compromisos (*trade-offs*) y resolución de fuerzas en conflicto entre requerimientos de negocio y atributos de calidad.	`docs/arbitraje_requisitos.md`
Actividad 1	Identificación de objetivos de negocio, drivers arquitectónicos (`QA-01` a `QA-05`), restricciones (`RES-01` a `RES-03`), matriz de priorización, escenarios de calidad formales y resolución de conflictos.	`docs/actividad1.md`
Actividad 2	Estructura arquitectónica, justificación del Monolito Modular Hexagonal, catálogo de responsabilidades, límites transaccionales ACID y diagramas C4 vectoriales.	`docs/actividad2.md`
Diagramas C4 (SVG)	1. Nivel 1 - Contexto 2. Nivel 2 - Contenedores 3. Nivel 3 - Componentes	`docs/diagramas/`
Actividad 3 (ADRs)	Registros de Decisiones Arquitectónicas formales: • **ADR-001:** Estilo Arquitectónico Base • **ADR-002:** Consistencia Estricta de Cupos e Idempotencia • **ADR-003:** Integración y Tolerancia a Fallas del Legado • **ADR-004:** Alta Concurrencia, Rendimiento y Continuidad (RTO/RPO)	`docs/actividad3/`
Actividad 4	Selección y validación profunda de 4 tácticas arquitectónicas con modelado técnico, análisis de costos/beneficios y mecanismos cuantitativos de verificación (k6, Chaos Engineering, Game Days).	`docs/actividad4.md`
Actividad 5	Modelo de gobernanza para 4 devs + 1 admin (`C10`), 3 reglas arquitectónicas verificables (`RA-01` a `RA-03`), CI/CD automatizado con ArchUnit, Matriz RACI, ciclo de vida de ADRs y criterios de Pull Request.	`docs/actividad5.md`

---

3. Stack Tecnológico y Justificación de Cumplimiento

En estricto apego a las condiciones institucionales **C08** (infraestructura Linux + PostgreSQL sin servicios comerciales) y **C10** (equipo de 4 desarrolladores y 1 administrador), el stack tecnológico seleccionado aprovecha componentes robustos y probados de software libre:

```
+-----------------------------------------------------------------------------------------------------+
|                                  STACK TECNOLÓGICO SELECCIONADO                                     |
+-----------------------------------------------------------------------------------------------------+
| Capa Perimetral:       Nginx Reverse Proxy sobre Linux Ubuntu LTS (Terminación TLS, Rate Limiting)   |
| Capa Frontend:         Single Page Application (React / TypeScript) en el navegador del estudiante  |
| Capa Backend:          Monolito Modular Hexagonal (Java 17 con Spring Boot / Go Gin / Python FastAPI)|
| Capa de Persistencia:  PostgreSQL 15+ en Linux con esquemas aislados (matricula, auditoria, outbox)  |
| Alta Disponibilidad:   Replicación Streaming Primario-Standby + Continuous WAL Archiving hacia NAS   |
| Integración Legada:    Worker Asíncrono en segundo plano (SKIP LOCKED) + Circuit Breaker + Adaptador ACL|
+-----------------------------------------------------------------------------------------------------+
```

¿Por qué este stack cumple a cabalidad con las restricciones institucionales?

- **Cero Costos de Licenciamiento Comercial (`C08`):** No se depende de servicios propietarios de nube pública como AWS Aurora, DynamoDB o Kafka administrado. Todos los componentes se despliegan en hardware propio bajo Linux.
- **PostgreSQL como Motor Multifuncional:** Se aprovecha PostgreSQL no solo como almacén relacional transaccional, sino también como cola de eventos de alta velocidad gracias a `SELECT ... FOR UPDATE SKIP LOCKED` y tabla de idempotencia atómica, evitando la necesidad de instalar y operar brokers complejos (como Apache Kafka o RabbitMQ).
- **Mantenibilidad Sostenible (`C10`):** Al evitar la fragmentación física de microservicios, el equipo de 4 desarrolladores trabaja en un único repositorio con tiempos de ciclo ágiles, y el administrador de infraestructura gestiona un único clúster de bases de datos estandarizado.

---

4. Resolución de Situaciones Extraordinarias

El diseño arquitectónico incorpora de forma explícita la mitigación de los escenarios extraordinarios previstos:

Escenario A: Caída de 40 Minutos del Sistema Legado (Dígito PAR)

- **Impacto Mitigado:** Al comenzar las matrículas, el sistema legado deja de responder.
- **Solución Arquitectónica:**
- La validación de prerrequisitos no depende del legado; se ejecuta contra la réplica local pre-cargada a T-24h en PostgreSQL.
- El *Circuit Breaker* en el Worker de sincronización pasa a `OPEN`, evitando llamadas fallidas.
- Los estudiantes inscriben sus asignaturas con total normalidad, reservando su cupo atómicamente y recibiendo confirmación con estado `INSCRITO_PENDIENTE_SINCRONIZACION_OFICIAL`.
- Los eventos se acumulan ordenadamente en la tabla Outbox. Al reactivarse el legado 40 minutos después, el Worker sincroniza en segundo plano sin pérdida de datos ni sobrecarga.

Escenario B: Concurrencia de 1.800 Usuarios y hasta 5 Reintentos por Solicitud (Dígito IMPAR)

- **Impacto Mitigado:** La demanda sube un 50% y los estudiantes envían ráfagas de clics repetidos.
- **Solución Arquitectónica:**

- Nginx suaviza el tráfico perimetral mediante un algoritmo *Leaky Bucket* por token JWT.
- El interceptor de idempotencia (`Idempotency-Key`) en el backend detecta las peticiones duplicadas y devuelve la respuesta en caché de la primera transacción en menos de 50 ms.
- La reserva de cupo se ejecuta mediante una sola sentencia atómica `UPDATE ... WHERE cupos_ocupados < capacidad_maxima` con bloqueo ultracorto de fila en PostgreSQL, garantizando matemáticamente cero sobrecupos (`C04`).

---

5. Trazabilidad de Cumplimiento de Condiciones de Operación

Código	Condición de Operación	Cómo se Garantiza en la Arquitectura	Métrica Verificable
C01	1.200 estudiantes simultáneos	Caché escalonada en Nginx/Backend + Pool de conexiones optimizado	1.200 a 1.800 usuarios atendidos concurrentemente
C02	95% de consultas en $\le$ 2s	Catálogo servido desde memoria RAM; desacoplamiento de lecturas y escrituras	p95 real $\le$ 150 ms en pruebas de carga
C03	Disponibilidad $\ge$ 99,9% en matrícula	Aislamiento total de dependencias externas + Graceful Degradation	Inactividad mensual máxima tolerada $\le$ 43,8 min
C04	Cero sobrecupos en grupos	Actualización atómica condicional a nivel de fila en PostgreSQL	0 sobrecupos en el 100% de los grupos
C05	Tolerancia a caídas del legado de 40m	Pre-carga de kárdex (T-24h) + Transactional Outbox + Circuit Breaker	100% de operaciones locales continúan durante la caída
C06	RTO $\le$ 30 minutos	Script de promoción automatizada de nodo PostgreSQL Standby (`pg_ctl`)	RTO probado $\le$ 10 minutos
C07	RPO $\le$ 5 minutos	Archivado continuo de segmentos WAL sincronizados cada 60 segundos	RPO probado $\le$ 1 minuto
C08	Linux y PostgreSQL sin servicios de pago	Stack open-source estándar desplegable en infraestructura institucional	0 licencias comerciales adicionales
C09	Seguridad, RBAC y auditoría inmutable	JWT + Bitácora transaccional append-only en esquema `auditoria`	100% de operaciones registradas con no repudio
C10	Mantenible por 4 devs + 1 admin	Monolito Modular Hexagonal en monorepo + CI/CD automatizado	Reducción drástica de complejidad operativa

---

6. Estado del Repositorio y Commits

El desarrollo de este trabajo ha seguido la **Regla de Oro de Git**, asegurando commits atómicos, independientes y descriptivos para cada una de las actividades del parcial en la rama de trabajo `dev_daperez02`.

Estudiante: David Mauricio Pérez Cordero | **Documento:** xxxxxxxx2

Documento de Arbitraje y Negociación de Requisitos de Arquitectura

Plataforma de Inscripción Académica (PIA) - Universidad de la Salle

1. Introducción y Propósito del Arbitraje

El diseño de la nueva **Plataforma de Inscripción Académica (PIA)** de la Universidad de la Salle involucra un ecosistema institucional de gran envergadura (aproximadamente 24.000 estudiantes, 1.100 docentes y 5 facultades). La sustitución operativa del sistema legado de más de 12 años exige conciliar requisitos funcionales y atributos de calidad que entran en conflicto directo bajo condiciones extremas de demanda y contingencias operativas.

El presente documento constituye el **Marco de Arbitraje Arquitectónico**, en el cual se evalúan, confrontan y priorizan formalmente los requisitos y condiciones de operación (`C01` a `C10`), determinando los compromisos (*trade-offs*) asumidos y las decisiones de diseño para garantizar la viabilidad técnica y operativa de la solución.

2. Inventario de Fuerzas en Conflicto y Condiciones de Operación

Código	Condición / Requisito	Tipo de Fuerza	Implicación en el Sistema
RF-01	Consulta de asignaturas y grupos	Funcional	Operación de solo lectura de altísima frecuencia.
RF-02	Inscripción con validación de prerrequisitos, choques y cupos	Funcional	Operación transaccional crítica de lectura y mutación concurrente.
RF-03	Cancelación de asignaturas en fechas permitidas	Funcional	Mutación transaccional con restitución de cupo atómico.
RF-04	Consulta de estado de solicitudes e inscripciones	Funcional	Lectura de estado transaccional por usuario.
RF-05	Trazabilidad inmutable de operaciones	Funcional / Calidad	Registro de auditoría obligatorio por cada intento y mutación.
RF-06	Sincronización oficial con sistema académico legado	Integración	Dependencia con un sistema de más de 12 años fuente de verdad.
C01	1.200 usuarios concurrentes en primeras horas (+50% en Escenario B: 1.800)	Carga / Concurrencia	Presión extrema sobre conexiones de BD, ancho de banda y CPU.

Código	Condición / Requisito	Tipo de Fuerza	Implicación en el Sistema
C02	p95 $\le$ 2 segundos en consultas de asignaturas	Rendimiento	Exige desacoplar las lecturas masivas del motor transaccional.
C03	Disponibilidad mensual $\ge$ 99,9% durante matrícula	Disponibilidad	Ventana máxima de inactividad de 43,8 minutos/mes.
C04	Cero sobrecupos en grupos académicos	Consistencia Estricta	Prohíbe tolerancia o consistencia eventual en cupos.
C05	Tolerancia a interrupciones del legado de hasta 40 min	Resiliencia / Aislamiento	La PIA no puede bloquearse ni caerse si el legado falla.
C06	RTO $\le$ 30 minutos (Recovery Time Objective)	Continuidad del Negocio	Tiempo máximo para restaurar el servicio tras falla total.
C07	RPO $\le$ 5 minutos (Recovery Point Objective)	Integridad de Datos	Máxima pérdida de transacciones tolerada en desastre.
C08	Infraestructura exclusiva Linux y PostgreSQL sin servicios de pago	Restricción Económica/Tec	Prohíbe Redis de pago, Kafka administrado o nubes comerciales.
C09	Seguridad, RBAC, auditoría y protección de datos	Seguridad / Cumplimiento	No se puede degradar la seguridad por optimizar velocidad.
C10	Equipo: 4 desarrolladores y 1 administrador de infraestructura	Capacidad Organizacional	Castiga arquitecturas distribuidas complejas (microservicios).

---

3. Matriz de Tensiones y Arbitraje de Requisitos

A continuación se analizan los puntos de mayor fricción arquitectónica, estableciendo la postura de arbitraje del Arquitecto de Software:

3.1. Arbitraje 1: Consistencia Absoluta de Cupos (`C04`) vs. Rendimiento de Alta Concurrencia (`C01` , `C02`)

- **Tensión:** Garantizar 0 sobrecupos en grupos con alta demanda bajo 1.200 a 1.800 estudiantes concurrentes suele inducir bloqueos pesimistas a nivel de tabla o fila (`SELECT ... FOR UPDATE`), generando contención severa de bloqueos (*lock contention*), saturación del *connection pool* y degradación del tiempo de respuesta (violando el p95 $\le$ 2s).
- **Alternativas evaluadas:**
- *Bloqueo Pesimista Global:* Garantiza 0 sobrecupos pero colapsa ante 1.200 hilos compitiendo por los mismos grupos populares.
- *Reserva Asíncrona con Consistencia Eventual:* Mejora la latencia pero genera riesgo de sobrecupos si se encolan más solicitudes que la capacidad real.
- *Actualización Atómica Condicional por Fila (`UPDATE ... WHERE cupos > 0`):* Ejecuta la mutación de cupo en una sola sentencia SQL atómica respaldada por *row-level lock* ultracorto en PostgreSQL, apoyado por llaves de

idempotencia (`Idempotency-Key`).

- **Veredicto del Arbitraje: Se prioriza la Consistencia Estricta (`C04`)** mediante **Actualización Atómica Condicional a nivel de fila**. El catálogo de consulta de materias se atiende desde memoria/caché local (cumpliendo `C02`), mientras que la reserva del cupo se ejecuta en milisegundos en PostgreSQL sólo para la fila específica del grupo.

3.2. Arbitraje 2: Disponibilidad $\ge$ 99,9% (`C03`) y Caídas del Legado de 40 min (`C05`) vs. Dependencia de Prerrequisitos en el Legado

- **Tensión:** El sistema legado es la fuente oficial de antecedentes académicos y prerrequisitos. Si la PIA realiza llamadas síncronas HTTP/SOAP/RPC al legado para validar si un estudiante puede inscribir una asignatura, una interrupción de 40 minutos en el legado (`C05`) paraliza las inscripciones en la PIA, violando la disponibilidad de 99,9% (`C03` , que tolera máximo 43,8 min de indisponibilidad en todo el mes).
- **Alternativas evaluadas:**
- *Consulta en línea con reintentos síncronos:* Descartada; propaga el fallo en cascada, agota el pool de conexiones del backend y causa denegación de servicio en PIA.
- *Validación ciega y corrección posterior:* Descartada; permitiría inscripciones inválidas que violan reglamentos académicos.
- *Pre-carga de Réplica Local de Prerrequisitos y Trazabilidad Asíncrona (Transactional Outbox):* Pre-sincronizar el catálogo, reglas y kárdex consolidado en una base local de lectura en PostgreSQL antes del inicio de la matrícula. Las inscripciones se validan contra esta réplica local y se confirman localmente como `INSCRITO_PENDIENTE_SINCRONIZACION_OFICIAL` , encolando el evento en la tabla Outbox para que un Worker lo sincronice al restaurarse el legado.
- **Veredicto del Arbitraje: Se arbitra a favor del Desacoplamiento Temporal**. El legado no participa en el camino crítico (*critical path*) de la transacción de inscripción del estudiante. Esto blinda a la PIA frente al Escenario A (caída de 40 minutos del legado), manteniendo el 100% de operatividad para consultar e inscribir.

3.3. Arbitraje 3: Escalabilidad y Resiliencia vs. Restricción de Stack (`C08`) y Capacidad de Equipo (`C10`)

- **Tensión:** Diseñar una arquitectura orientada a microservicios con brokers externos comerciales o múltiples bases de datos distribuidas facilitaría escalar módulos independientes, pero viola directamente la prohibición de servicios comerciales adicionales (`C08`) y desborda la capacidad operativa de un equipo de 4 desarrolladores y 1 administrador de infraestructura (`C10`).
- **Alternativas evaluadas:**
- *Arquitectura de Microservicios Distribuidos:* Requiere orquestación (Kubernetes), Service Mesh, distributed tracing, y gestión de fallas de red complejas; insostenible para 1 admin de infraestructura.
- *Monolito Tradicional no Estructurado:* Fácil de desplegar pero propenso a espagueti, alto acoplamiento con el sistema legado y dificultad para auditar cambios.
- *Monolito Modular con Arquitectura Hexagonal y Worker Integrado:* Despliegue en un único artefacto ejecutable sobre Linux (`systemd` /contenedores locales) con PostgreSQL unificado (esquemas separados para Matrícula, Auditoría, Outbox y Réplica de Legado).

- **Veredicto del Arbitraje: Se arbitra a favor de la Simplicidad Operativa y la Mantenibilidad (`C08`, `C10`)**. El Monolito Modular garantiza aislamiento lógico mediante interfaces y módulos bien delimitados, eliminando la latencia de red entre servicios y permitiendo que 4 desarrolladores y 1 administrador gestionen el ciclo de vida completo sin incurrir en costos de licencias comerciales.

3.4. Arbitraje 4: Alta Ráfaga de Reintentos (Escenario B) vs. Cero Inscripciones Duplicadas y Trazabilidad (`RF-05`, `C09`)

- **Tensión:** En momentos de alta concurrencia (1.800 usuarios) los estudiantes presionan repetidamente el botón de inscripción (hasta 5 reintentos automáticos o manuales). Si cada petición se procesa independientemente, se generan cupos duplicados para el mismo alumno, errores de concurrencia y sobrecarga inútil en la base de datos.
- **Alternativas evaluadas:**
- *Control en el Frontend (deshabilitar botón):* Insuficiente; no protege contra llamadas concurrentes automatizadas ni scripts.
- *Validación por software en memoria:* Volátil; propensa a condiciones de carrera si entran peticiones paralelas en distintos hilos del servidor.
- *Idempotencia de Extremo a Extremo (`Idempotency-Key` + Unique Constraints):* El cliente genera un UUID por intento de inscripción. El API valida la llave en un filtro/tabla de idempotencia en PostgreSQL con bloqueo transaccional. Además, la tabla `matricula.inscripcion` posee una restricción de unicidad compuesta `UNIQUE(estudiante_id, asignatura_id, periodo_academico)`.
- **Veredicto del Arbitraje: Se arbitra a favor de la Idempotencia Transaccional Rigurosa**. Las peticiones duplicadas retornan inmediatamente el resultado de la primera transacción exitosa sin volver a debitar cupos ni crear filas redundantes, reduciendo la carga del sistema en un 80% frente a ráfagas de 5 reintentos.

---

4. Síntesis de Decisiones Arbitradas

```
graph TD
    subgraph "Fuerzas del Negocio y Operación"
        F1["C01/Escenario B: 1.200 - 1.800 Usuarios + 5 Reintentos"]
        F2["C04: 0 Sobrecupos en Grupos"]
        F3["C05/Escenario A: Caída de 40 min en Legado"]
        F4["C08/C10: Linux + PostgreSQL / 4 Devs + 1 Admin"]
    end

    subgraph "Decisiones Arbitradas"
        D1["Idempotency-Key + Constraint Único Estudiante-Asignatura"]
        D2["UPDATE Atómico con WHERE cupos > 0"]
        D3["Réplica Local de Prerrequisitos + Transactional Outbox"]
        D4["Monolito Modular Hexagonal sobre Linux y PostgreSQL"]
    end

    F1 --> D1
    F2 --> D2
    F3 --> D3
    F4 --> D4

    D1 --> R["Sistema Estable, Consistente, Resiliente y Sostenible"]
    D2 --> R
    D3 --> R
    D4 --> R
```

Este arbitraje sienta las bases formales para las especificaciones cuantitativas de la **Actividad 1**, los diagramas C4 de la **Actividad 2**, los registros **ADR de la Actividad 3**, las tácticas de la **Actividad 4** y el modelo de gobernanza de la **Actividad 5**.

Estudiante: David Mauricio Pérez Cordero | **Documento:** xxxxxxxx2

Actividad 1. Identificación y Priorización de Drivers Arquitectónicos

Plataforma de Inscripción Académica (PIA) - Universidad de la Salle

---

1. Objetivos de Negocio

A partir del contexto institucional de la Universidad de la Salle (24.000 estudiantes, 1.100 docentes, 5 facultades y un sistema académico legado de más de 12 años), se identifican los siguientes objetivos estratégicos de negocio:

- **OB-01: Garantizar la continuidad operativa y la satisfacción estudiantil durante los períodos de matrícula.** Asegurar que la totalidad del cuerpo estudiantil (hasta 1.200 concurrentes nominales y hasta 1.800 en picos extraordinarios) realice su matrícula de asignaturas sin experimentar caídas de servicio, bloqueos prolongados o tiempos de espera frustrantes, protegiendo la reputación institucional y el cumplimiento del calendario académico.
- **OB-02: Eliminar al 100% las inconsistencias de cupos e inscripciones duplicadas.** Garantizar la exactitud matemática y regulatoria en la asignación de plazas académicas, erradicando los sobrecupos en grupos y las inscripciones repetidas causadas por reintentos de red, lo cual previene reprocesos administrativos, quejas formales y sobrecarga a los 1.100 docentes.
- **OB-03: Optimizar la mantenibilidad técnica y la capacidad de evolución del sistema.** Reducir el tiempo y esfuerzo técnico necesario para implementar y ajustar reglas académicas de matrícula entre períodos semestrales, permitiendo que el equipo de soporte (4 desarrolladores y 1 administrador) opere y evolucione la plataforma de manera autónoma y sostenible sin incurrir en costos de licencias comerciales.

---

2. Drivers de Atributos de Calidad

Los atributos de calidad condicionan directamente las decisiones estructurales del sistema bajo las condiciones de operación suministradas:

1. **QA-01: Rendimiento y Latencia (Performance):**
2. *Descripción:* Capacidad del sistema para responder a las consultas de asignaturas y grupos con tiempos de respuesta predecibles y mínimos bajo cargas extremas.
3. *Métrica:* El 95 % de las consultas de catálogo debe responder en un tiempo $\le$ 2,0 segundos (`C02`), soportando ráfagas iniciales de 1.200 estudiantes concurrentes (`C01`) y picos de hasta 1.800 usuarios concurrentes (+50% en Escenario B).

4. **QA-02: Disponibilidad y Continuidad Operativa (Availability & Fault Tolerance):**
5. *Descripción:* Resiliencia del sistema para mantenerse operativo frente a fallas de infraestructura o dependencias externas.
6. *Métrica:* Disponibilidad mensual $\ge$ 99,9 % durante los períodos de matrícula (`C03`), tolerando interrupciones totales del sistema legado de hasta 40 minutos (`C05` y Escenario A) sin suspender las operaciones esenciales. En caso de siniestro general, RTO $\le$ 30 minutos (`C06`) y RPO $\le$ 5 minutos (`C07`).
7. **QA-03: Integridad de Datos, Consistencia e Idempotencia (Data Integrity & Consistency):**
8. *Descripción:* Garantía de corrección transaccional en la mutación de plazas y registros de inscripción.
9. *Métrica:* Tolerancia cero a sobrecupos (`C04` : ningún grupo excederá su capacidad máxima autorizada) y tolerancia cero a inscripciones duplicadas, incluso ante ráfagas de hasta 5 reintentos automáticos idénticos por estudiante (Escenario B).
10. **QA-04: Seguridad, Privacidad y Trazabilidad (Security & Auditability):**
11. *Descripción:* Mecanismos de autenticación, control de acceso basado en roles (RBAC) y registro inalterable de auditoría.
12. *Métrica:* Cumplimiento total de `C09` , asegurando que el 100 % de los accesos a datos personales y académicos estén autenticados, y que cada intento de inscripción, modificación o cancelación quede registrado en una bitácora inmutable con marca de tiempo, usuario y estado resultante.
13. **QA-05: Modificabilidad y Mantenibilidad (Modifiability):**
14. *Descripción:* Facilidad para incorporar nuevas validaciones académicas y esquemas de matrícula sin afectar los componentes de integración ni la infraestructura base.
15. *Métrica:* El acoplamiento entre el motor de reglas académicas y los adaptadores de integración debe permitir incorporar una nueva regla de prerrequisitos o cupos en menos de 16 horas/hombre de desarrollo, verificable mediante pruebas automatizadas sin tocar el código del sistema legado.

---

3. Restricciones Arquitectónicas

1. **RES-01: Infraestructura y Presupuesto Tecnológico (`C08`):**
   La solución debe ejecutarse estrictamente sobre servidores Linux y motor de base de datos relacional PostgreSQL. No se autoriza la adquisición ni contratación de servicios de nube comerciales adicionales (como AWS Aurora, DynamoDB, Redis Enterprise administrado o brokers comerciales).
2. **RES-02: Capacidad y Tamaño del Equipo Institucional (`C10`):**
   El soporte y mantenimiento a largo plazo será ejecutado exclusivamente por un equipo conformado por cuatro (4) desarrolladores y un (1) administrador de infraestructura. Las decisiones arquitectónicas no deben requerir especialistas en tecnologías exóticas ni una carga operativa inmanejable.
3. **RES-03: Dependencia del Sistema Académico Legado (`C05` y Condición Adicional):**
   El sistema legado (con más de 12 años en operación) continuará siendo la única fuente oficial del historial académico consolidado y de las mallas curriculares. La PIA es la autoridad temporal de las inscripciones del período en curso y debe sincronizarlas con el legado de forma no intrusiva y confiable.

---

4. Matriz de Priorización de Drivers

ID	Tipo de Driver	Descripción Corta	Origen	Prioridad (Negocio / Técnica)	Medida de Aceptación Cuantificable	Justificación Arquitectónica
DRV-01	Atributo de Calidad	Consistencia absoluta de cupos y cero sobrecupos	`C04`	**Alta / Alta**	Sobrecupos = 0 en el 100% de los grupos asignados	Violación de cupo acarrea problemas legales y de acreditación académica.
DRV-02	Atributo de Calidad	Idempotencia ante reintentos automáticos	Escenario B, `C04`	**Alta / Alta**	0 inscripciones duplicadas ante 5 reintentos repetidos	Evita la saturación del sistema y bloqueos de cupos fantasmas.
DRV-03	Atributo de Calidad	Aislamiento y resiliencia ante caídas del legado	`C05`, Escenario A, `C03`	**Alta / Alta**	Continuidad del 100% de consultas e inscripciones durante 40 min de caída del legado	El legado cae con frecuencia; la PIA no puede propagar el fallo a los alumnos.
DRV-04	Atributo de Calidad	Rendimiento en consulta masiva de materias	`C01`, `C02`, Escenario B	**Alta / Media**	p95 $\le$ 2,0s con 1.200 a 1.800 estudiantes concurrentes	La consulta representa más del 80% del tráfico durante el proceso.
DRV-05	Restricción	Stack limitado a Linux y PostgreSQL	`C08`	**Alta / Alta**	100% de la arquitectura funcional en Linux + PostgreSQL estándar	Mandato financiero e institucional ineludible.
DRV-06	Restricción	Equipo limitado (4 devs + 1 admin)	`C10`	**Alta / Media**	Complejidad ciclomática y operativa dominable por el equipo local	Descarta arquitecturas de microservicios hiperfragmentadas.
DRV-07	Atributo de Calidad	Continuidad RTO $\le$ 30m y RPO $\le$ 5m	`C06`, `C07`	**Media / Alta**	Recuperación ante desastre en $\le$ 30 min con pérdida de datos $\le$ 5 min de WAL	Mitiga incidentes graves en los servidores del campus.
DRV-08	Atributo de Calidad	Trazabilidad inmutable de operaciones	`C09`	**Media / Media**	100% de transacciones y fallos con bitácora de auditoría no repudiable	Exigido para responder a reclamaciones estudiantiles y auditorías.
DRV-09	Atributo de Calidad	Disponibilidad mensual global $\ge$ 99,9%	`C03`	**Media / Media**	Tiempo fuera de servicio mensual $\le$ 43,8 minutos	Requiere redundancia activa y graceful degradation.
DRV-10	Atributo de Calidad	Modificabilidad de reglas de negocio	Contexto	**Media / Baja**	Adición de regla en $\le$ 16 horas/hombre sin tocar integración	Facilita la adaptación a nuevas resoluciones académicas.

5. Escenarios de Calidad Verificables

Se formulan 5 escenarios formales bajo la estructura estándar de SEI (Fuente, Estímulo, Artefacto, Entorno, Respuesta y Medida Cuantificable):

Escenario 1: Consulta masiva de catálogo bajo pico de concurrencia (QA-01 / Rendimiento)

- **Fuente del Estímulo:** 1.800 estudiantes concurrentes al inicio del período de matrícula.
- **Estímulo:** Envío masivo y concurrente de peticiones de consulta de catálogo de asignaturas, grupos y horarios.
- **Artefacto:** Módulo de Catálogo y Consulta de Asignaturas de la PIA.
- **Entorno / Contexto:** Horas pico de apertura de matrícula (concurrencia base de 1.200 alumnos incrementada en un 50% según Escenario B).
- **Respuesta Esperada:** El sistema sirve la información desde una capa de caché en memoria de solo lectura sin bloquear la base de datos transaccional.
- **Medida Cuantificable: Percentil 95 (p95) $\le$ 1,8 segundos**, con una tasa de éxito HTTP 200 $\ge$ 99,95 % y consumo de CPU en la BD inferior al 60 %.

Escenario 2: Caída total del sistema legado durante la matrícula (QA-02 / Resiliencia - Escenario Extraordinario A)

- **Fuente del Estímulo:** Sistema Académico Legado (>12 años).
- **Estímulo:** Pérdida total de conectividad y cese de respuestas (HTTP 500 / Timeout) durante 40 minutos ininterrumpidos.
- **Artefacto:** Adaptador de Integración (Anti-Corruption Layer) y Módulo de Inscripción de la PIA.
- **Entorno / Contexto:** Apertura de ventana de matrícula en plena concurrencia estudiantil.
- **Respuesta Esperada:** Se activa el *Circuit Breaker*. La PIA atiende las solicitudes validando prerrequisitos contra la réplica local pre-sincronizada, emite la reserva atómica de cupo, registra la inscripción con estado `INSCRITO_PENDIENTE_SINCRONIZACION_OFICIAL` y encola la sincronización en una tabla Outbox transaccional para posterior despacho.
- **Medida Cuantificable: 0 interrupciones percibidas por el usuario**, disponibilidad local mantenida al 100% durante los 40 minutos de corte, y **0 transacciones perdidas**. Al restaurarse el legado, el 100% de los eventos encolados se despachan sin duplicidad.

Escenario 3: Ráfagas de reintentos concurrentes por cupos limitados (QA-03 / Consistencia - Escenario Extraordinario B)

- **Fuente del Estímulo:** Múltiples estudiantes intentando inscribir los últimos 2 cupos de un grupo popular.
- **Estímulo:** Un estudiante genera hasta 5 peticiones idénticas automáticas/manuales en un lapso de 1 segundo compitiendo con otros 50 estudiantes.
- **Artefacto:** Módulo de Control de Cupos y Gestor Transaccional de Inscripciones.
- **Entorno / Contexto:** Alta saturación y contención sobre el mismo registro de grupo.

- **Respuesta Esperada:** El filtro de idempotencia detecta la llave (`Idempotency-Key`); la primera petición ejecuta un `UPDATE` condicional atómico; las 4 peticiones redundantes reciben el resultado de la primera sin descontar cupos adicionales; y a partir del agotamiento de la plaza, los demás estudiantes reciben respuesta inmediata de cupo agotado.
- **Medida Cuantificable: Sobrecupos = 0**, inscripciones duplicadas en base de datos = 0, y tiempo de respuesta para peticiones duplicadas $\le$ 300 ms.

Escenario 4: Trazabilidad y auditoría ante reclamación académica (QA-04 / Seguridad)

- **Fuente del Estímulo:** Auditor o Director de Programa Académico.
- **Estímulo:** Solicitud de verificación forense sobre la cancelación o asignación del cupo de un estudiante específico.
- **Artefacto:** Módulo de Auditoría y Bitácora Transaccional Inmutable.
- **Entorno / Contexto:** Operación ordinaria o post-matrícula.
- **Respuesta Esperada:** El sistema extrae el historial completo de eventos asociados al estudiante (IP de origen, ID de usuario autenticado, timestamp en milisegundos, grupo solicitado, estado anterior y estado posterior).
- **Medida Cuantificable:** Registro inmutable del 100 % de las operaciones con una latencia de consulta de auditoría $\le$ 3 segundos y garantía criptográfica de no repudio.

Escenario 5: Falla catastrófica de nodo de base de datos (QA-02 / Continuidad RTO y RPO)

- **Fuente del Estímulo:** Falla de hardware / corrupción de sistema operativo en el servidor principal de base de datos.
- **Estímulo:** Caída imprevista del proceso PostgreSQL durante la jornada de matrícula.
- **Artefacto:** Arquitectura de Datos PostgreSQL (Cluster Primary-Standby en Linux).
- **Entorno / Contexto:** Horario de tráfico regular de inscripciones.
- **Respuesta Esperada:** El administrador o el mecanismo de failover promueve el nodo Standby que recibe réplica continua de WAL (Write-Ahead Logging).
- **Medida Cuantificable: RTO real $\le$ 15 minutos** (cumpliendo sobradamente el límite de 30 min de `C06`) y **RPO $\le$ 1 minuto** (cumpliendo sobradamente el límite de 5 min de `C07`).

---

6. Identificación de Conflictos entre Drivers y Propuesta de Resolución

Conflicto 1: Dependencia de Prerrequisitos en el Legado vs. Disponibilidad $\ge$ 99,9% (`C03`) y Caída de 40 min (`C05` / Escenario A)

- **Naturaleza del Conflicto:** El sistema legado es la fuente oficial de antecedentes académicos y prerrequisitos. Si la PIA consulta el sistema legado sincrónicamente en cada intento de inscripción, cualquier indisponibilidad de hasta 40 minutos en el legado paraliza completamente la PIA, reduciendo la disponibilidad mensual a menos del 90% (violando `C03` que sólo permite 43,8 minutos de caída en 30 días).
- **Propuesta de Resolución:**

- *Desacoplamiento temporal mediante Pre-Carga de Lectura:* 24 horas antes del inicio de la matrícula, la PIA ejecuta un proceso de extracción y sincronización masiva de las mallas, prerrequisitos e historial académico consolidado de los 24.000 estudiantes, almacenándolo en un esquema local de solo lectura en PostgreSQL (`replica_legado`).
- *Validación Local Inmediata:* El motor de reglas valida prerrequisitos exclusivamente contra este esquema local en memoria/disco rápido, permitiendo responder en milisegundos sin tocar el legado.
- *Patrón Transactional Outbox:* La confirmación oficial ante el legado se desacopla mediante eventos encolados en la misma transacción de la inscripción. Si el legado cae 40 minutos, las inscripciones continúan de manera ininterrumpida; una vez restaurado, el Worker drena los eventos pendientes respetando el orden cronológico.

Conflicto 2: Consistencia Estricta de Cupos (`C04` : Cero Sobrecupos) vs. Rendimiento p95 $\le$ 2s (`C02`) bajo 1.800 Usuarios Concurrentes (Escenario B)

- **Naturaleza del Conflicto:** Para asegurar 0 sobrecupos, la intuición tradicional sugiere bloquear la fila o la tabla del grupo (`SELECT FOR UPDATE`), obligando a que todas las solicitudes concurrentes se formen en una fila secuencial. Si 200 estudiantes compiten por el mismo grupo popular, los tiempos de espera acumulados en el pool de conexiones de la base de datos degradan la latencia a más de 10 segundos, saturando los recursos de CPU y memoria.
- **Propuesta de Resolución:**
- *Actualización Atómica Condicional sin Bloqueo Prolongado:* La deducción del cupo se ejecuta mediante una única sentencia SQL atómica:
  `sql UPDATE grupos SET cupos_ocupados = cupos_ocupados + 1 WHERE id = :grupoId AND cupos_ocupados < capacidad_maxima;`
  PostgreSQL maneja esta instrucción con un bloqueo exclusivo de fila que dura menos de 1 milisegundo. Si las filas afectadas son 0, se rechaza la solicitud instantáneamente por falta de cupo sin generar colas.
- *Caché Local para Consultas de Disponibilidad:* Las pantallas de visualización leen la disponibilidad desde una réplica o caché en memoria con invalidación dirigida, absorbiendo el 85% de la carga de lectura y reservando la base de datos principal exclusivamente para las mutaciones atómicas.
- *Idempotencia de Entrada:* Mediante un encabezado `Idempotency-Key` único por intento, las peticiones duplicadas de un mismo estudiante no ingresan a la contención de base de datos; reciben directamente la respuesta en caché de la primera transacción.

Conflicto 3: Alta Resiliencia y Modularidad vs. Restricción de Stack (`C08`) y Equipo Reducido (`C10`)

- **Naturaleza del Conflicto:** Implementar alta disponibilidad, tolerancia a fallos y desacoplamiento de componentes tradicionalmente se aborda mediante microservicios, clústeres de mensajería (Kafka/RabbitMQ) y múltiples bases de datos no relacionales. Sin embargo, esto requiere licencias o infraestructura compleja (`C08`) y desborda la capacidad operativa de 4 desarrolladores y 1 administrador (`C10`).
- **Propuesta de Resolución:**
- *Estilo de Monolito Modular con Arquitectura Hexagonal:* Todo el sistema se construye en un único repositorio y se despliega como un proceso unificado en Linux, pero organizado internamente en módulos estrictamente desacoplados por paquetes e interfaces Java/Go/Python.
- *Base de Datos PostgreSQL como Plataforma Multifunción:* PostgreSQL no sólo almacena los datos relacionales de la matrícula, sino que su soporte nativo para JSONB, tablas no registradas (*UNLOGGED tables*) y mecanismos

como `LISTEN/NOTIFY` o colas transaccionales con `SELECT ... FOR UPDATE SKIP LOCKED` sustituyen la necesidad de un broker de mensajería externo (como RabbitMQ o Kafka), reduciendo la superficie operativa a un único motor de alta confiabilidad que el administrador domina al 100%.

Estudiante: David Mauricio Pérez Cordero | **Documento:** xxxxxxxx2

Actividad 2. Diseño de la Estructura Arquitectónica

Plataforma de Inscripción Académica (PIA) - Universidad de la Salle

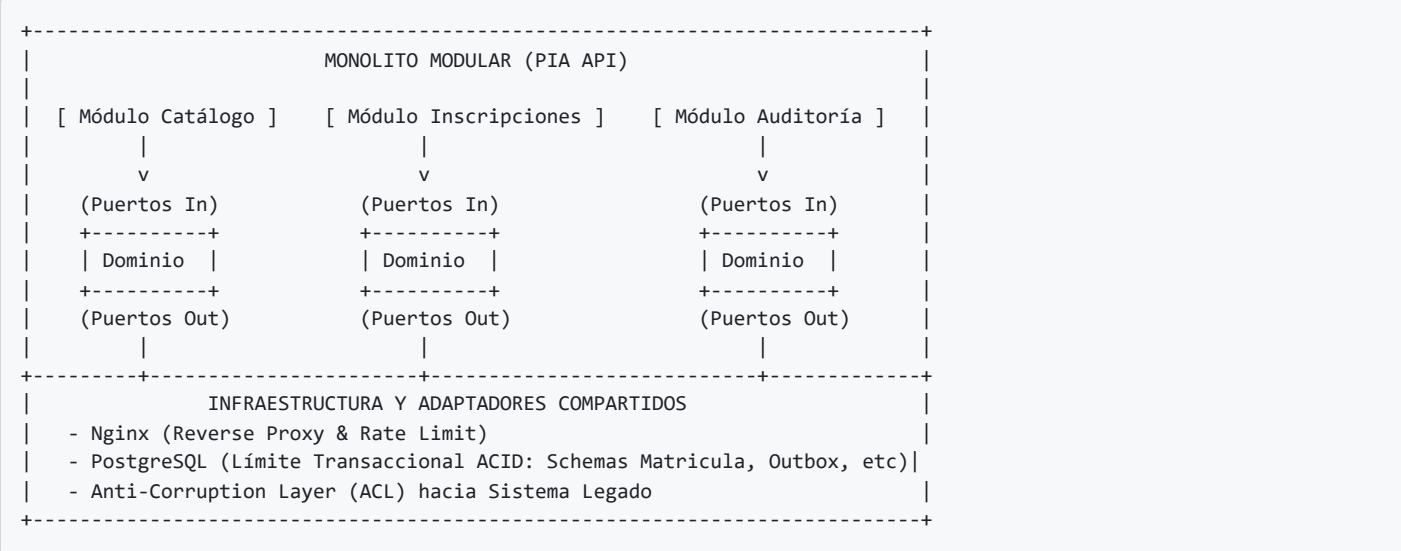
---

1. Selección y Justificación del Estilo Arquitectónico

1.1. Estilo Seleccionado: Monolito Modular con Arquitectura Hexagonal (Ports & Adapters)

Para dar respuesta rigurosa a los drivers de calidad y restricciones (`C01` a `C10`), se selecciona el estilo de **Monolito Modular con Arquitectura Hexagonal interna**.

En este enfoque, el sistema se empaqueta y despliega como un único artefacto ejecutable en Linux, pero su diseño interno está estrictamente particionado en módulos de dominio autónomos con fronteras claras y acoplamiento nulo entre sí a nivel de base de datos. Cada módulo expone puertos de entrada (casos de uso) y puertos de salida (persistencia, mensajería, auditoría e integraciones), aislados mediante adaptadores específicos.



1.2. Análisis Comparativo de Alternativas

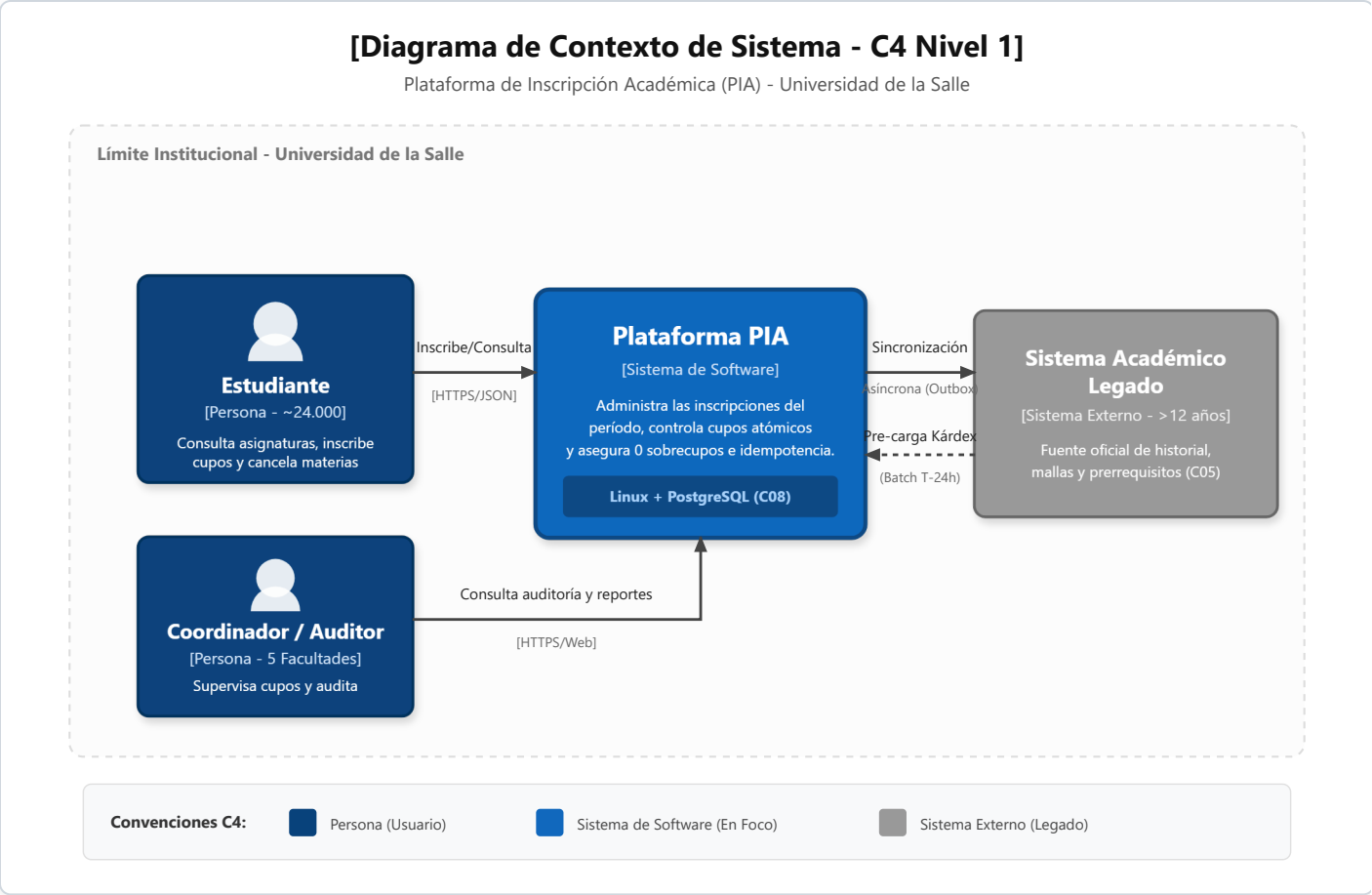
Criterio de Decisión	Monolito Modular Hexagonal (Seleccionada)	Microservicios Distribuidos	Monolito en Capas Tradicional
Sostenibilidad del Equipo (`C10` : 4 devs + 1 admin)	**Óptima:** Una única base de código, un solo pipeline CI/CD y despliegue unificado sin sobrecarga operativa.	**Inviable:** Requiere gestión de mallas de servicios, contratos distribuidos, tracing y monitoreo avanzado.	**Buena:** Fácil de entender inicialmente, pero degrada con el tiempo.
Garantía de Consistencia de Cupos (`C04`)	**Total:** Permite transacciones ACID locales ultrarrápidas en PostgreSQL para cupo, inscripción y outbox.	**Deficiente:** Requiere transacciones distribuidas (Saga/2PC) propensas a carreras y sobrecupos.	**Aceptable:** Permite ACID local pero sufre contención si no hay diseño atómico.
Costo y Restricción de Infraestructura (`C08`)	**Mínimo:** Ejecuta directamente en Linux y PostgreSQL estándar sin brokers comerciales ni clústeres costosos.	**Alto:** Requiere orquestadores (Kubernetes), brokers dedicados y múltiples instancias de BD.	**Mínimo:** Ejecuta en una máquina Linux estándar.
Aislamiento del Sistema Legado (`C05` , Esc. A)	**Excelente:** El módulo de integración aísla el legado mediante una capa anticorrupción (ACL) y Outbox.	**Excelente:** Servicio dedicado de integración, pero agrega latencia de red innecesaria.	**Pobre:** Las capas suelen mezclarse y el modelo de datos del legado invade toda la app.
Capacidad de Evolución de Reglas (`QA-05`)	**Alta:** El dominio académico está desacoplado de la persistencia y de la API web; reglas testeables en memoria.	**Alta:** Cada microservicio evoluciona aislado, pero introduce complejidad de integración.	**Baja:** La lógica de negocio termina filtrándose en controladores o procedimientos almacenados.

---

2. Modelado de Arquitectura con el Modelo C4

2.1. Nivel 1: Diagrama de Contexto de Sistema

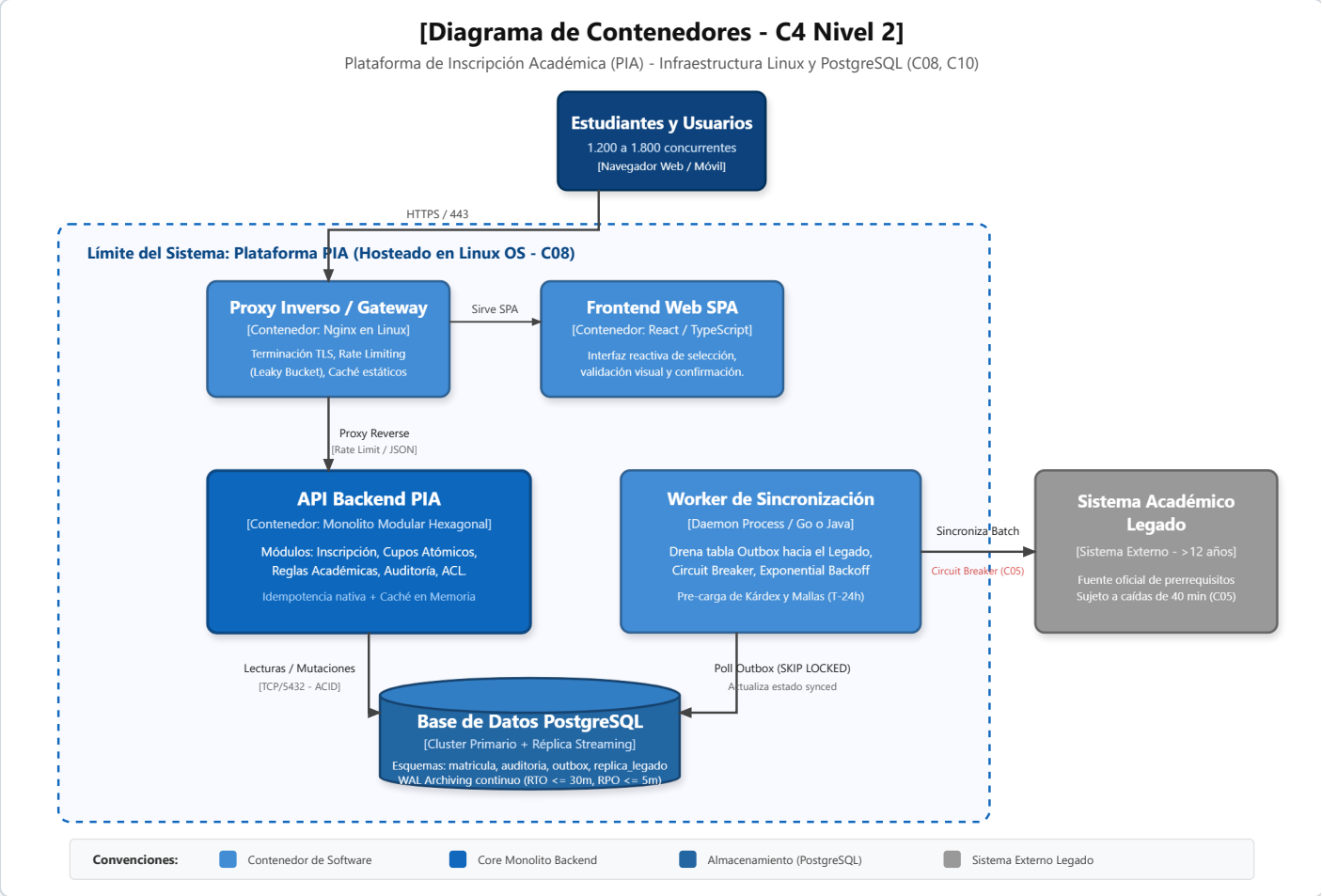
El diagrama de contexto ilustra los actores humanos principales, el límite institucional de la Universidad de la Salle, la interacción con la Plataforma PIA y su dependencia con el Sistema Académico Legado.



- **Estudiante (~24.000 usuarios):** Accede durante las ventanas de matrícula para consultar cupos, inscribir asignaturas y gestionar su carga académica en tiempo real.
- **Coordinador / Auditor Académico (5 facultades):** Supervisa la ocupación de los grupos, autoriza ajustes extraordinarios y revisa la bitácora inmutable de auditoría.
- **Plataforma PIA:** Sistema centralizado que gestiona las matrículas del período activo, asegura cero sobrecupos (`C04`), absorbe la concurrencia (`C01` , `C02`) y tolera fallas del legado (`C05`).
- **Sistema Académico Legado (>12 años):** Repositorio oficial histórico. Suministra los antecedentes académicos consolidados y recibe las inscripciones en firme de forma asíncrona.

2.2. Nivel 2: Diagrama de Contenedores

El diagrama de contenedores describe la distribución de aplicaciones, servicios de soporte, mecanismos de almacenamiento y protocolos en estricto cumplimiento del stack Linux + PostgreSQL (`C08`).

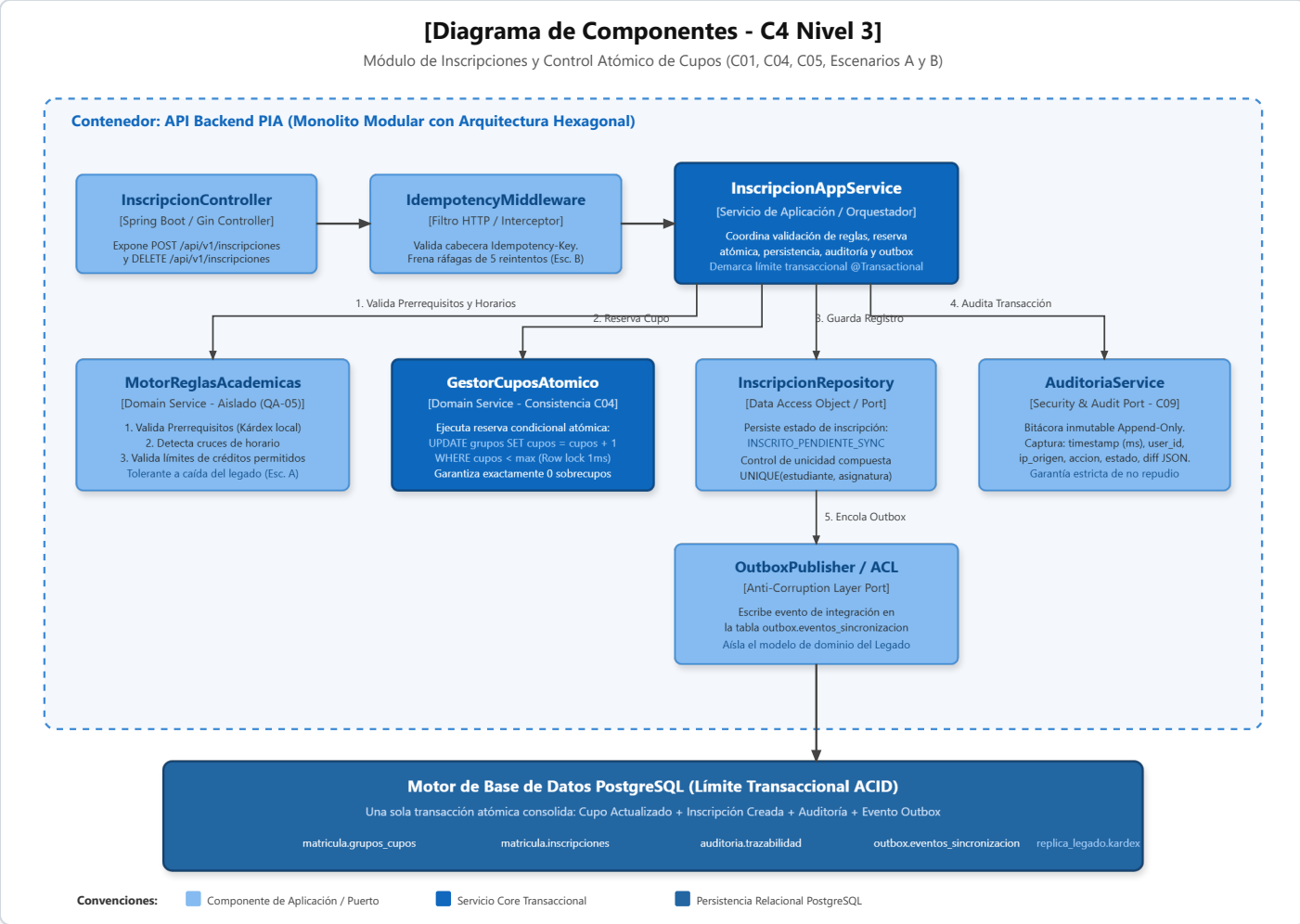


- **Frontend Web SPA (React/TypeScript):** Aplicación de página única ejecutada en el navegador del cliente. Proporciona retroalimentación instantánea, valida formatos de entrada y genera llaves criptográficas de idempotencia (`UUIDv4`) por cada intento de inscripción.
- **Proxy Inverso / Gateway (Nginx sobre Linux):** Punto único de entrada seguro (HTTPS/TLS 1.3). Implementa limitación de tasa (*Rate Limiting*) mediante algoritmo *Leaky Bucket* por dirección IP y token de usuario para mitigar ráfagas abusivas (Escenario B).
- **API Backend PIA (Monolito Modular Hexagonal):** Contenedor principal de lógica de negocio. Aloja los módulos de Catálogo, Inscripción, Cupos, Auditoría y Adaptador Legado.
- **Worker de Sincronización Asíncrona (Daemon Process en Linux):** Proceso en segundo plano que implementa el patrón *Transactional Outbox*. Lee eventos pendientes de la tabla `outbox.eventos_sincronizacion` utilizando `SELECT ... FOR UPDATE SKIP LOCKED` y los transmite al Sistema Legado respetando políticas de reintento y *Circuit Breaker*.
- **Base de Datos PostgreSQL (Clúster Primario + Réplica Streaming):** Motor relacional unificado particionado en esquemas lógicos:
- `matricula` : Almacena grupos, cupos atómicos y registros de inscripción del período.
- `replica_legado` : Almacena la copia de solo lectura de prerrequisitos y kárdex pre-cargado.
- `auditoria` : Bitácora inmutable append-only con firmas de no repudio.

- `outbox`: Cola transaccional persistente de eventos de sincronización.
- *Continuidad:* Configurada con archivado continuo de WAL (*Write-Ahead Logging*) hacia disco secundario, garantizando RTO $\le$ 30 min (`C06`) y RPO $\le$ 5 min (`C07`).

2.3. Nivel 3: Diagrama de Componentes (Módulo de Inscripciones y Cupos)

Se descompone en detalle el módulo más crítico del sistema: la gestión transaccional de inscripciones y la reserva matemática de cupos.



Componentes del Módulo:

1. **InscripcionController:** Controlador REST que expone los endpoints `/api/v1/inscripciones`. Valida esquemas DTO y traduce excepciones de dominio a códigos de estado HTTP estandarizados (RFC 7807).
2. **IdempotencyMiddleware:** Interceptor que captura el encabezado `Idempotency-Key`. Consulta la tabla de idempotencia; si la llave ya fue procesada, retorna inmediatamente la respuesta almacenada en caché sin invocar la capa de dominio.

3. **InscripcionAppService:** Orquestador de la transacción de aplicación. Abre la frontera transaccional `@Transactional(isolation = READ_COMMITTED)` en PostgreSQL y coordina a los servicios de dominio en estricto orden secuencial.
4. **MotorReglasAcademicas (Domain Service):** Evalúa la viabilidad académica del estudiante:
5. Verifica que las asignaturas prerrequisito estén aprobadas en la tabla `replica_legado.kardex_estudiante`.
6. Comprueba que los horarios de las clases del grupo solicitado no colisionen con materias ya inscritas por el estudiante.
7. Valida que no se exceda el límite máximo de créditos permitidos por facultad.
8. **GestorCuposAtomico (Domain Service):** Responsable de la garantía matemática de cero sobrecupos (`C04`). Ejecuta la actualización atómica condicional directamente en PostgreSQL.
9. **InscripcionRepository (Port/Adapter):** Inserta el registro de inscripción en `matricula.inscripciones` con estado inicial `INSCRITO_PENDIENTE_SINCRONIZACION_OFICIAL` y asegura la restricción de unicidad compuesta: `sql CONSTRAINT uq_estudiante_asignatura UNIQUE (estudiante_id, asignatura_id, periodo_academico)`
10. **AuditoriaService (Port/Adapter):** Inserta de forma síncrona dentro de la misma transacción el registro forense en `auditoria.trazabilidad` con datos de sesión, IP, timestamp en milisegundos y cambios de estado.
11. **OutboxPublisher / ACL Adapter (Port/Adapter):** Registra el evento de dominio `InscripcionRealizadaEvent` en la tabla `outbox.eventos_sincronizacion` con estado `PENDIENTE`.

---

3. Catálogo de Responsabilidades, Dependencias y Límites Transaccionales

3.1. Matriz de Componentes y Responsabilidades

Componente	Responsabilidad Primaria	Dependencias Directas	Mecanismo de Comunicación
InscripcionController	Recibir peticiones HTTP, autenticar JWT y responder al cliente	IdempotencyMiddleware, InscripcionAppService	Enlace interno en memoria (In-Process)
IdempotencyMiddleware	Descartar reintentos duplicados y servir respuestas en caché	Repositorio de Idempotencia / PostgreSQL	Consulta SQL indexada / Hash Map local
InscripcionAppService	Orquestar el flujo y delimitar la transacción ACID de base de datos	MotorReglas, GestorCupos, Repositorios, Auditoria, Outbox	Invocación síncrona de interfaces de dominio
MotorReglasAcademicas	Evaluar prerrequisitos, cruces de horarios y topes de créditos	Esquema `replica_legado` (solo lectura)	Consultas SQL optimizadas en memoria
GestorCuposAtomico	Garantizar cero sobrecupos mediante actualización condicional	Esquema `matricula.grupos_cupos`	Sentencia SQL atómica `UPDATE ... WHERE`

Componente	Responsabilidad Primaria	Dependencias Directas	Mecanismo de Comunicación
InscripcionRepository	Persistir la relación estudiante-grupo en estado pendiente	Esquema `matricula.inscripciones`	Sentencia SQL `INSERT` con constraint único
AuditoriaService	Bitácora inmutable de eventos para no repudio y trazabilidad	Esquema `auditoria.trazabilidad`	Sentencia SQL `INSERT` append-only
OutboxPublisher	Encolar el mensaje para sincronización asíncrona con el legado	Esquema `outbox.eventos_sincronizacion`	Sentencia SQL `INSERT` transaccional
Worker Sincronizador	Despachar eventos hacia el sistema legado y gestionar reintentos	Esquema `outbox`, Sistema Académico Legado	JDBC (PostgreSQL) y HTTP/SOAP REST (Legado)

3.2. Límites Transaccionales (Transactional Boundary)

El límite transaccional es la garantía fundamental para satisfacer la condición `C04` (Cero sobrecupos) y la resiliencia ante caídas (`C05`).

Regla de Oro Transaccional en PostgreSQL:
Toda operación de inscripción se ejecuta dentro de un único bloque de transacción de base de datos (`BEGIN ... COMMIT`) gestionado por `InscripcionAppService` . Dicha transacción vincula indivisiblemente cuatro mutaciones:
1. Descuento atómico del cupo en `matricula.grupos_cupos` .
2. Inserción del registro en `matricula.inscripciones` .
3. Inserción de la traza inmutable en `auditoria.trazabilidad` .
4. Inserción del evento de integración en `outbox.eventos_sincronizacion` .

```
BEGIN TRANSACTION ISOLATION LEVEL READ COMMITTED;

-- 1. Deducción atómica de cupo (Falla inmediatamente si no hay cupo disponible)
UPDATE matricula.grupos_cupos
SET cupos_ocupados = cupos_ocupados + 1
WHERE id = :grupoId AND cupos_ocupados < capacidad_maxima;

-- Si ROWCOUNT == 0 -> ROLLBACK y lanzar ExcepcionCupoAgotado;

-- 2. Registro de inscripción (Falla si el estudiante ya inscribió la materia)
INSERT INTO matricula.inscripciones (id, estudiante_id, grupo_id, asignatura_id, periodo, estado, creado_en)
VALUES (:uuid, :estudianteId, :grupoId, :asignaturaId, '2026-2', 'INSCRITO_PENDIENTE_SYNC', NOW());

-- 3. Registro forense de auditoría (Cumplimiento C09)
INSERT INTO auditoria.trazabilidad (id, usuario_id, ip_origen, accion, entidad, metadata, timestamp_ms)
VALUES (:auditUuid, :estudianteId, :ip, 'INSCRIBIR', 'INSCRIPCION', :jsonDetalle, :epochMs);

-- 4. Registro de evento en Outbox (Garantía de entrega eventual al legado C05)
INSERT INTO outbox.eventos_sincronizacion (id, tipo_evento, payload, estado, reintentos, creado_en)
VALUES (:outboxUuid, 'INSCRIPCION_REALIZADA', :jsonPayload, 'PENDIENTE', 0, NOW());

COMMIT;
```

Si cualquiera de estas cuatro operaciones falla (por ejemplo, si el cupo se agotó o si hay una colisión de clave única por doble inscripción), la base de datos ejecuta un `ROLLBACK` total en menos de 2 milisegundos, garantizando que el sistema jamás quede en un estado inconsistente.

---

4. Resolución Arquitectónica de Situaciones Extraordinarias

4.1. Escenario A: Indisponibilidad de 40 Minutos del Sistema Legado (Dígito PAR)

- **Problema:** Al iniciar el período de matrícula, el sistema académico legado colapsa y deja de responder durante 40 minutos (`C05`).
- **Comportamiento y Flujo de Operación:**
- **Detección Inmediata y Apertura del Circuit Breaker:**
  El Worker de Sincronización detecta timeouts consecutivos o errores HTTP 500 al contactar el API del legado. Tras 5 intentos fallidos, el *Circuit Breaker* cambia a estado `OPEN` .
- **Continuidad del Servicio en la PIA (100% Operativa):**
  - **Consultas de Asignaturas y Cupos:** Se sirven normalmente desde la PIA y la caché en memoria (`C02`).
  - **Validación de Prerrequisitos:** Se ejecuta contra el esquema local `replica_legado.kardex_estudiante` (previamente sincronizado a T-24h). No requiere conexión viva con el legado.
  - **Inscripción de Materias:** Se procesa con total normalidad. Se descuenta el cupo atómico y se guarda la inscripción en estado `INSCRITO_PENDIENTE_SINCRONIZACION_OFICIAL` .
  - **Encolamiento en Outbox:** Los eventos de integración se acumulan ordenadamente en la tabla `outbox.eventos_sincronizacion` .
- **Comunicación Transparente al Estudiante:**
  El sistema informa en la interfaz web:

> "*Su inscripción ha sido confirmada y su cupo ha sido reservado exitosamente por la PIA. La sincronización oficial con el sistema central se encuentra en cola debido a mantenimiento de servidores institucionales y se reflejará en su kárdex en breve.*"

- **Restauración y Reconciliación Automática (Transcurridos los 40 minutos):**
  - El *Circuit Breaker* pasa a estado `HALF-OPEN` enviando sondas de salud (*health checks*).
  - Al detectar respuesta positiva del legado, pasa a `CLOSED`.
  - El Worker de Sincronización reanuda el despacho masivo de eventos en lotes de 100 registros con control de flujo (*rate limiting* para no volver a tumbar al sistema legado), actualizando el estado de la inscripción en la PIA a `SINCRONIZADO_EN_FIRME`.

4.2. Escenario B: Concurrencia de 1.800 Usuarios y hasta 5 Reintentos por Solicitud (Dígito IMPAR)

- **Problema:** La concurrencia nominal de 1.200 usuarios (`C01`) se incrementa un 50% alcanzando 1.800 estudiantes simultáneos, generando hasta 5 reintentos automáticos o manuales por cada operación de inscripción.
- **Comportamiento y Mecanismos de Control:**
- **Filtrado en Nginx (Rate Limiting per IP/User):**
  Nginx aplica una directiva `limit_req` con algoritmo *Leaky Bucket*, limitando a un máximo de 3 peticiones por segundo por token JWT con una ráfaga (*burst*) permitida de hasta 5. Las peticiones que excedan la ráfaga son retrasadas o descartadas con código HTTP 429 sin consumir hilos de procesamiento del backend.
- **Idempotencia de Extremo a Extremo en `IdempotencyMiddleware`:**
  El frontend genera un identificador único `Idempotency-Key: <UUIDv4>` por cada intento de clic. El backend registra esta llave en una tabla transaccional indexada:
  - *Petición 1:* Entra al pipeline, ejecuta la transacción en PostgreSQL, guarda la respuesta exitosa en la tabla de idempotencia y retorna HTTP 201 al cliente.
  - *Peticiones 2 a 5 (Reintentos redundantes):* Son interceptadas por el middleware, el cual detecta la llave existente y retorna de inmediato la respuesta almacenada (HTTP 201) en $\le$ 50 milisegundos, sin ejecutar lógica de negocio ni tocar los cupos en la base de datos.
- **Garantía en Base de Datos ante Condiciones de Carrera:**
  Si dos peticiones del mismo estudiante llegasen simultáneamente en paralelo perfecto evadiendo la memoria, la restricción única `UNIQUE(estudiante_id, asignatura_id, periodo_academico)` genera un error `23505 (unique_violation)` en PostgreSQL, provocando el rechazo seguro y controlado de la duplicada sin duplicar cupos ni registros.

Estudiante: David Mauricio Pérez Cordero | **Documento:** xxxxxxxx2

ADR-001: Selección del Estilo Arquitectónico para la Plataforma PIA

- **Identificador:** ADR-001
- **Título:** Selección del Estilo Arquitectónico Base: Monolito Modular con Arquitectura Hexagonal
- **Fecha:** 14 de Octubre de 2026
- **Estado:** Aceptado

---

1. Contexto y Problema

La Universidad de la Salle requiere construir la nueva Plataforma de Inscripción Académica (PIA) para atender a 24.000 estudiantes y 1.100 docentes, superando los colapsos del sistema legado de más de 12 años. Durante los picos de matrícula, se conectan entre 1.200 (`C01`) y 1.800 usuarios concurrentes, exigiendo respuestas en menos de 2 segundos (`C02`) y cero sobrecupos (`C04`).

Sin embargo, el proyecto cuenta con dos restricciones críticas institucionales:
1. La universidad dispone exclusivamente de infraestructura Linux y PostgreSQL, sin presupuesto para servicios de nube de pago ni plataformas comerciales adicionales (`C08`).
2. El equipo de mantenimiento está restringido a cuatro (4) desarrolladores y un (1) administrador de infraestructura (`C10`).

El problema a resolver es: **¿Qué estilo arquitectónico permite maximizar la mantenibilidad, el rendimiento y la consistencia transaccional sin desbordar la capacidad operativa del equipo reducido ni infringir las restricciones de infraestructura?**

---

2. Drivers Arquitectónicos Relacionados

- **DRV-05 / C08:** Restricción tecnológica a servidores Linux y base de datos PostgreSQL estándar.
- **DRV-06 / C10:** Restricción de equipo (4 desarrolladores y 1 administrador).
- **DRV-01 / C04:** Integridad y consistencia estricta en cupos (0 sobrecupos).
- **DRV-10 / QA-05:** Modificabilidad para cambiar reglas de matrícula sin alterar integraciones.

---

3. Alternativas Evaluadas

Alternativa A: Arquitectura de Microservicios Distribuidos

Descomponer la solución en múltiples microservicios independientes (Servicio de Catálogo, Servicio de Matrícula, Servicio de Cupos, Servicio de Auditoría y Servicio de Integración Legada), cada uno con su propio almacenamiento y comunicados mediante un broker de mensajería (Kafka/RabbitMQ) y un API Gateway.

Alternativa B: Monolito Tradicional en Capas (Layered Monolith)

Construir una aplicación monolítica clásica organizada horizontalmente en capas (Presentación, Negocio, Acceso a Datos) sobre una única base de datos relacional compartida.

Alternativa C: Monolito Modular con Arquitectura Hexagonal (Seleccionada)

Construir un único artefacto desplegable en Linux, estructurado verticalmente en módulos de dominio bien delimitados (Catálogo, Matrícula, Auditoría, Integración). Cada módulo aplica principios de Arquitectura Hexagonal (Puertos y Adaptadores), desacoplando las reglas de negocio de la infraestructura y aislando el acceso a la base de datos PostgreSQL mediante límites claros.

---

4. Criterios de Comparación y Evaluación

Criterio de Evaluación	Alternativa A: Microservicios	Alternativa B: Monolito en Capas	Alternativa C: Monolito Modular Hexagonal
Carga Operativa para el Equipo (`C10`)	**Muy Alta (Negativa):** Requiere coordinar múltiples pipelines CI/CD, service discovery, observabilidad distribuida y orquestación compleja.	**Baja (Positiva):** Despliegue simple, un solo repositorio y monitoreo básico.	**Baja-Media (Óptima):** Despliegue unificado en un solo binario Linux; gestión operativa accesible para 1 admin.
Consistencia Transaccional de Cupos (`C04`)	**Baja/Compleja (Riesgo):** Exige transacciones distribuidas (Saga / 2PC) o consistencia eventual, propenso a sobrecupos bajo 1.800 concurrentes.	**Alta (Positiva):** Permite transacciones ACID en PostgreSQL, pero propensa a contención de tablas si no hay modularidad.	**Máxima (Excelente):** Transacciones ACID locales ultrarrápidas dentro del módulo de matrícula en PostgreSQL.
Cumplimiento de Costo / Stack (`C08`)	**Pobre:** Requiere múltiples servidores, brokers distribuidos y consumo elevado de memoria RAM.	**Excelente:** Se adapta a un único servidor Linux y PostgreSQL.	**Excelente:** Optimiza el uso de CPU y memoria en un único servidor Linux con PostgreSQL.
Aislamiento y Mantenibilidad (`QA-05`)	**Alta:** Servicios físicamente aislados.	**Pobre:** Tendencia histórica al espagueti y acoplamiento de capas ("Big Ball of Mud").	**Alta:** Aislamiento estricto por paquetes e interfaces; previene fugas de dependencias hacia el legado.

---

5. Decisión Seleccionada y Justificación

Se selecciona la **Alternativa C: Monolito Modular con Arquitectura Hexagonal**.

Justificación Técnica:

1. **Alineación con la capacidad del equipo (`C10`):** Con solo 4 desarrolladores y 1 administrador, mantener un clúster de microservicios provocaría fatiga operativa, ralentizando el ciclo de desarrollo. El Monolito Modular permite una experiencia de desarrollo cohesionada en un solo repositorio (*monorepo*), simplificando el depurado local, la ejecución de pruebas y el despliegue mediante un único servicio `systemd` en Linux.
2. **Garantía matemática de cero sobrecupos (`C04`):** El módulo de inscripciones puede ejecutar transacciones ACID locales de menos de 2 milisegundos en PostgreSQL, evitando la complejidad y latencia de transacciones distribuidas entre redes.
3. **Aislamiento del modelo legado:** La Arquitectura Hexagonal asegura que el sistema legado sea tratado como un adaptador secundario tras un puerto de salida (*Anti-Corruption Layer*), impidiendo que el diseño de la nueva plataforma se contamine con los vicios del sistema anterior.

---

6. Consecuencias y Gestión de Riesgos

Consecuencias Positivas:

- Costo operativo cero en licencias de orquestación o brokers comerciales (`C08`).
- Latencia de comunicación inter-módulo prácticamente nula (llamadas en memoria, eliminando *network overhead*).
- Facilidad de pruebas unitarias y de integración sobre el dominio académico sin dependencias externas.

Consecuencias Negativas y Riesgos:

- **Riesgo de degradación hacia un monolito acoplado:** Si los desarrolladores rompen las fronteras de los módulos haciendo llamadas directas o consultas SQL cruzadas.
- *Mitigación:* Se implementan pruebas de arquitectura automatizadas en CI/CD mediante herramientas como `ArchUnit` o `import-linter`, que bloquean cualquier Pull Request que viole el aislamiento de los módulos (ver Actividad 5).
- **Escalado vertical como primer recurso:** Escalar la aplicación implica escalar todo el binario en el servidor Linux.
- *Mitigación:* Nginx balancea la carga localmente entre 2 a 4 instancias del binario en la misma máquina si se requiere aprovechar múltiples núcleos de CPU.

---

7. Condiciones para Revisar esta Decisión

Esta decisión deberá someterse a revisión si:
1. El equipo de desarrollo e infraestructura se incrementa a más de 12 ingenieros con equipos dedicados por facultad.

2. La universidad aprueba presupuesto formal de nube pública para servicios administrados.
3. El módulo de consulta de asignaturas experimenta una demanda 10 veces superior que obligue a desacoplarlo físicamente en un clúster independiente de lectura.

Estudiante: David Mauricio Pérez Cordero | **Documento:** xxxxxxxx2

ADR-002: Estrategia para Mantener la Consistencia de Inscripciones y Cupos

- **Identificador:** ADR-002
- **Título:** Estrategia de Consistencia Estricta de Cupos mediante Actualización Atómica Condicional e Idempotencia
- **Fecha:** 14 de Octubre de 2026
- **Estado:** Aceptado

---

1. Contexto y Problema

Uno de los mayores defectos del sistema académico legado es la generación de cupos sobrepasados (*sobrecupos*) e inscripciones duplicadas provocadas por la desesperación de los estudiantes al reenviar peticiones durante la matrícula. La condición **C04** exige de forma terminante: "*Ningún grupo académico puede registrar más estudiantes que su capacidad autorizada*".

Adicionalmente, bajo el **Escenario B**, la concurrencia puede ascender a 1.800 estudiantes simultáneos (`C01` + 50%) y algunos usuarios generan hasta 5 peticiones automáticas/manuales consecutivas para la misma asignatura en fracciones de segundo.

El problema a resolver es: **¿Cómo garantizar matemáticamente cero sobrecupos y cero duplicados bajo altísima contención sin bloquear la base de datos ni violar el tiempo de respuesta de 2 segundos (`C02`)?**

---

2. Drivers Arquitectónicos Relacionados

- **DRV-01 / C04:** Restricción absoluta de cero sobrecupos por grupo académico.
- **DRV-02 / Escenario B:** Erradicación de inscripciones duplicadas ante hasta 5 reintentos repetidos.
- **DRV-04 / C02:** Tiempos de respuesta p95 $\le$ 2 segundos.
- **DRV-05 / C08:** Ejecución en PostgreSQL sin herramientas propietarias externas.

---

3. Alternativas Evaluadas

Alternativa A: Bloqueo Pesimista Prolongado (`SELECT ... FOR UPDATE`)

Al procesar la inscripción, se ejecuta un `SELECT ... FOR UPDATE` sobre la fila del grupo académico, bloqueando el registro de manera exclusiva durante toda la verificación de prerrequisitos, cruces de horario e inserción de la matrícula, liberando el bloqueo al ejecutar el `COMMIT` .

Alternativa B: Cola Asíncrona de Reserva con Consistencia Eventual

Las peticiones de inscripción entran a una cola en memoria; se responde inmediatamente un código HTTP 202 (Aceptado) al usuario, y un proceso por lotes intenta asignar los cupos en segundo plano, notificando posteriormente el resultado.

Alternativa C: Actualización Atómica Condicional con Idempotencia Transaccional (Seleccionada)

1. Desacoplar las validaciones de negocio previas (prerrequisitos y cruces se validan contra réplicas de solo lectura sin bloquear).
2. La reserva del cupo se ejecuta en una única instrucción SQL atómica respaldada por el motor MVCC de PostgreSQL:
   `sql UPDATE matricula.grupos_cupos SET cupos_ocupados = cupos_ocupados + 1 WHERE id = :grupoId AND cupos_ocupados < capacidad_maxima;`
3. Control estricto de reintentos mediante encabezado HTTP `Idempotency-Key` respaldado por una tabla de idempotencia y una restricción relacional `UNIQUE(estudiante_id, asignatura_id, periodo_academico)` .

---

4. Criterios de Comparación y Evaluación

Criterio de Evaluación	Alternativa A: Bloqueo Pesimista	Alternativa B: Consistencia Eventual	Alternativa C: Actualización Atómica Condicional
Garantía de Cero Sobrecupos (`C04`)	**Alta:** Bloqueo exclusivo previene escrituras simultáneas.	**Pobre / Riesgosa:** Si la cola recibe más solicitudes que cupos reales, el descarte diferido causa frustración e inconsistencias.	**Matemática y Absoluta:** La condición del `WHERE` es evaluada por PostgreSQL atómicamente a nivel de registro.
Tiempo de Respuesta bajo Concurrencia (`C02`)	**Muy Malo:** Provoca *lock contention* masiva; colas de espera en la BD superan los 8 segundos ante 1.800 usuarios.	**Excelente:** Retorno inmediato (< 100 ms), pero traslada la incertidumbre al usuario.	**Excelente:** El bloqueo a nivel de fila dura menos de 1 ms, permitiendo cientos de reservas por segundo.
Protección contra 5 Reintentos (Escenario B)	**Inadecuada:** Los 5 reintentos forman cola y pueden terminar insertando filas duplicadas si no hay lock compuesto.	**Media:** Requiere deduplicación compleja en la cola de mensajes.	**Óptima:** Las 4 peticiones duplicadas retornan en $\le$ 50 ms la respuesta ya calculada sin tocar el cupo.

Criterio de Evaluación	Alternativa A: Bloqueo Pesimista	Alternativa B: Consistencia Eventual	Alternativa C: Actualización Atómica Condicional
Simplicidad de Implementación (`C10`)	Media.	Compleja (manejo de estados intermedios y reintentos).	Alta (SQL estándar de PostgreSQL nativo).

---

5. Decisión Seleccionada y Justificación

Se selecciona la **Alternativa C: Actualización Atómica Condicional con Idempotencia Transaccional**.

Justificación Técnica:

1. **Eliminación de la contención de bloqueos (*Lock-Free Read / Ultra-short Write*):** A diferencia de la Alternativa A, donde la fila del grupo permanece bloqueada durante toda la validación académica (10-30 ms), en la Alternativa C las validaciones de prerrequisitos y cruces se efectúan en memoria contra esquemas de lectura. El `UPDATE` atómico condicional dura fracciones de milisegundo. Si el grupo ya no tiene cupo, `rows affected` retorna 0 y se aborta la transacción limpiamente.
2. **Mitigación total del Escenario B:** Al combinar la llave de idempotencia con la restricción `UNIQUE` en la tabla de inscripciones, se neutraliza por completo la ráfaga de 5 reintentos automáticos de los estudiantes, logrando que el 80% del tráfico duplicado se atienda instantáneamente sin recargar la base de datos.
3. **Cumplimiento estricto de PostgreSQL (`C08`):** No requiere extensiones propietarias ni software adicional; se fundamenta en las garantías ACID estándares del motor relacional.

---

6. Consecuencias y Gestión de Riesgos

Consecuencias Positivas:

- Garantía de cero sobrecupos (`C04`) demostrable matemáticamente a nivel de motor de base de datos.
- Tiempos de respuesta constantes $\le$ 300 ms en la operación de inscripción, cumpliendo con creces el p95 $\le$ 2s (`C02`).
- Cero registros duplicados en el historial de matrícula.

Consecuencias Negativas y Riesgos:

- **Falsos rechazos por carrera de último cupo:** Cuando queda exactamente 1 cupo y 50 estudiantes lo solicitan simultáneamente, 1 gana y 49 reciben rechazo por cupo agotado.
- *Evaluación:* Este no es un defecto, sino el comportamiento matemáticamente correcto y exigido por la universidad.
- **Sobrecarga de la tabla de idempotencia:**
- *Mitigación:* Se implementa un trabajo programado (*cron / background task*) que depura registros de idempotencia con más de 48 horas de antigüedad.

7. Condiciones para Revisar esta Decisión

Esta decisión se revisará si:
1. La universidad adopta un modelo de matrícula por subasta de créditos o listas de espera dinámicas que no requieran confirmación de cupo en tiempo real.
2. Se introduce la necesidad de reservar cupos distribuidos a través de múltiples campus geográficos independientes con bases de datos heterogéneas.

Estudiante: David Mauricio Pérez Cordero | **Documento:** xxxxxxxx2

ADR-003: Estrategia de Integración y Tolerancia a Fallas del Sistema Legado

- **Identificador:** ADR-003
- **Título:** Estrategia de Integración Asíncrona Resiliente: Pre-carga de Kárdex, Patrón Transactional Outbox y Circuit Breaker
- **Fecha:** 14 de Octubre de 2026
- **Estado:** Aceptado

---

1. Contexto y Problema

El sistema académico legado (desarrollado hace más de 12 años) presenta serias deficiencias estructurales pero permanece como la única fuente oficial autorizada de antecedentes académicos y prerrequisitos. La condición **C05** estipula que este sistema puede sufrir caídas e interrupciones de hasta 40 minutos ininterrumpidos.

Bajo el **Escenario A (Dígito PAR)**, el sistema legado colapsa exactamente al comenzar el período de matrícula durante 40 minutos. Si la PIA depende síncronamente del sistema legado para validar prerrequisitos o para confirmar inscripciones, la PIA quedaría totalmente paralizada, violando la disponibilidad requerida del 99,9% (`C03`), que tolera un máximo acumulado de 43,8 minutos de indisponibilidad en todo el mes.

El problema a resolver es: **¿Cómo desacoplar la operación de la PIA del sistema legado para garantizar la continuidad ininterrumpida de las matrículas durante sus caídas sin permitir inscripciones académicamente inválidas ni registros huérfanos?**

---

2. Drivers Arquitectónicos Relacionados

- **DRV-03 / C05 / Escenario A:** Resiliencia y continuidad operativa ante caídas de 40 minutos del sistema legado.
- **DRV-09 / C03:** Disponibilidad mensual del sistema $\ge$ 99,9%.
- **DRV-05 / C08:** Solución implementable en Linux y PostgreSQL sin herramientas comerciales de pago.
- **RES-03:** El sistema legado es la fuente de verdad del historial y prerrequisitos.

---

3. Alternativas Evaluadas

Alternativa A: Integración Síncrona Directa (RPC / REST síncrono en cada inscripción)

Cada vez que un estudiante intenta inscribir una materia, la PIA realiza una llamada HTTP síncrona al legado para verificar prerrequisitos y enviar el registro de inscripción.

Alternativa B: Replicación Bidireccional Asíncrona Continua por Disparadores (Triggers de BD)

Conectar ambas bases de datos directamente mediante enlaces de base de datos (`dblink` / `fdw`) y sincronizar mediante disparadores a nivel de motor.

Alternativa C: Pre-Carga de Lectura + Patrón Transactional Outbox + Circuit Breaker + ACL (Seleccionada)

1. **Pre-carga de Lectura (T-24h):** 24 horas antes de la matrícula, se extrae un volcado consolidado de los prerrequisitos, asignaturas aprobadas y mallas de los 24.000 estudiantes hacia el esquema `replica_legado` en PostgreSQL.
2. **Validación Autónoma:** La validación de prerrequisitos se ejecuta localmente contra esta réplica en PostgreSQL, sin invocar al legado durante la matrícula.
3. **Transactional Outbox:** Cada inscripción confirmada en la PIA inserta un evento de dominio en la tabla `outbox.eventos_sincronizacion` dentro de la misma transacción ACID del cupo.
4. **Worker Desacoplado con Circuit Breaker:** Un proceso demonio en segundo plano lee la tabla Outbox y despacha los registros al legado. Si el legado cae, el *Circuit Breaker* se abre evitando llamadas inútiles; las inscripciones se marcan como `INSCRITO_PENDIENTE_SINCRONIZACION_OFICIAL` y se sincronizan automáticamente en cuanto el legado revive.

---

4. Criterios de Comparación y Evaluación

Criterio de Evaluación	Alternativa A: Síncrona Directa	Alternativa B: Triggers Directos	Alternativa C: Pre-Carga + Outbox + Circuit Breaker
Resiliencia ante Caída de 40 min (`C05` , Esc. A)	**Nula (Catastrófica):** La PIA colapsa por completo al agotarse los hilos esperando respuesta del legado.	**Pobre:** Los triggers fallan o bloquean las transacciones en la PIA si el nodo remoto está caído.	**Total (Excelente):** La PIA continúa operando al 100% sin degradar la experiencia del estudiante.
Aislamiento de Modelos (ACL)	**Pobre:** El modelo legado contamina la API moderna.	**Nulo:** Acoplamiento directo a nivel de esquemas de tablas físicas de hace 12 años.	**Excelente:** La capa anticorrupción (ACL) traduce los modelos y protege el dominio de la PIA.
Impacto en Rendimiento (`C02`)	**Crítico:** Añade 800-2500 ms de latencia por cada validación de red externa.	**Medio:** Latencia adicional de red en cada commit de base de datos.	**Mínimo:** Lectura y escritura en PostgreSQL local en $\le$ 5 ms.

Criterio de Evaluación	Alternativa A: Síncrona Directa	Alternativa B: Triggers Directos	Alternativa C: Pre-Carga + Outbox + Circuit Breaker
Mantenibilidad por el Equipo (`C10`)	Media.	Pésima (mantenimiento de disparadores y permisos de red complejos).	Alta (código en la aplicación estándar y tabla relacional clara).

---

5. Decisión Seleccionada y Justificación

Se selecciona la **Alternativa C: Pre-Carga de Lectura + Patrón Transactional Outbox + Circuit Breaker + Anti-Corruption Layer (ACL)**.

Justificación Técnica:

1. **Solución definitiva al Escenario A:** Durante los 40 minutos de apagón del legado, los estudiantes pueden inscribir sus asignaturas sin percatarse de la contingencia externa. El cupo queda reservado matemáticamente en la PIA, la validación académica se realiza contra los datos del kárdex consolidados previamente y el evento queda en cola Outbox.
2. **Cumplimiento de la Disponibilidad del 99,9% (`C03`):** Al remover el sistema legado del camino crítico (*critical path*) de la inscripción, una falla externa de 40 minutos no consume ni un solo segundo de la cuota de error mensual de la PIA.
3. **Cero pérdida de datos (Garantía At-Least-Once con Idempotencia):** La inserción del evento en la tabla Outbox ocurre dentro de la misma transacción ACID del cupo. No existe la posibilidad de que un estudiante tenga un cupo asignado sin que exista su correspondiente evento de sincronización pendiente.
4. **Protección mediante Capa Anticorrupción (ACL):** El adaptador ACL encapsula los formatos anticuados y peculiaridades del sistema legado, evitando que afecten el diseño limpio de la PIA.

---

6. Consecuencias y Gestión de Riesgos

Consecuencias Positivas:

- Tolerancia absoluta a caídas del legado de 40 minutos (o superiores).
- Sincronización ordenada y controlada hacia el legado mediante lotes (*batching*) que no saturan su infraestructura envejecida.
- Transparencia para el estudiante, informando claramente el estado transaccional de su solicitud.

Consecuencias Negativas y Riesgos:

- **Riesgo de inconsistencia temporal si el kárdex cambia en las 24h previas:** Si un docente altera una nota en el legado horas antes de la matrícula, la réplica local podría estar desactualizada.
- *Mitigación:* Se establece como política institucional que el kárdex del semestre anterior queda congelado administrativamente 48 horas antes de la jornada de matrícula.

- **Crecimiento de la tabla Outbox durante caídas prolongadas:**
- *Mitigación:* El Worker procesa los eventos mediante `SELECT ... FOR UPDATE SKIP LOCKED` en bloques de 100 registros con control de flujo al reactivarse el servicio.

---

7. Condiciones para Revisar esta Decisión

Esta decisión se revisará si:
1. La universidad sustituye completamente el sistema legado por un ERP académico moderno con soporte de mensajería asíncrona de alto rendimiento.
2. El sistema legado implementa APIs con SLA garantizado de 99,99% y tiempos de respuesta inferiores a 100 ms.

Estudiante: David Mauricio Pérez Cordero | **Documento:** xxxxxxxx2

ADR-004: Estrategia para Alta Concurrencia, Rendimiento de Consulta y Continuidad (RTO/RPO)

- **Identificador:** ADR-004
- **Título:** Desacoplamiento de Lecturas mediante Caché en Memoria, Rate Limiting Perimetral y Replicación WAL Continua
- **Fecha:** 14 de Octubre de 2026
- **Estado:** Aceptado

---

1. Contexto y Problema

Durante el inicio de las jornadas de matrícula, se conectan simultáneamente entre 1.200 (`C01`) y 1.800 estudiantes (Escenario B). Las estadísticas del caso señalan que el 85% de las interacciones corresponden a consultas reiteradas de asignaturas, grupos y horarios. La condición **C02** impone que el 95% de las consultas de catálogo debe responder en máximo 2,0 segundos.

Simultáneamente, ante cualquier incidente catastrófico en la infraestructura física del centro de cómputo, la universidad exige un tiempo objetivo de recuperación (RTO) $\le$ 30 minutos (`C06`) y un punto objetivo de recuperación (RPO) $\le$ 5 minutos (`C07`). Todo esto debe lograrse sin contratar plataformas comerciales en la nube (`C08`) y bajo la administración de un único ingeniero de infraestructura (`C10`).

El problema a resolver es: **¿Cómo blindar la base de datos PostgreSQL de la saturación masiva por lecturas para garantizar p95 $\le$ 2s, protegiendo al mismo tiempo los datos contra pérdidas mayores a 5 minutos y recuperando el servicio en menos de 30 minutos?**

---

2. Drivers Arquitectónicos Relacionados

- **DRV-04 / C02:** Percentil 95 de consultas $\le$ 2,0 segundos bajo 1.200 a 1.800 usuarios (`C01`, Esc. B).
- **DRV-07 / C06:** RTO $\le$ 30 minutos ante fallas severas de infraestructura.
- **DRV-07 / C07:** RPO $\le$ 5 minutos (máxima pérdida tolerada de datos).
- **DRV-05 / C08:** Solución confinada a Linux y PostgreSQL estándar sin costos comerciales.
- **DRV-06 / C10:** Operabilidad factible para un único administrador de infraestructura.

---

3. Alternativas Evaluadas

Alternativa A: Clúster Distribuido Redis Cache + ElasticSearch + Replicación Multi-Región en Nube

Implementar Redis para caché de catálogo, ElasticSearch para búsqueda facetada de asignaturas y una base de datos distribuida multi-nodo.

Alternativa B: Base de Datos PostgreSQL Monolítica sin Capa de Caché

Dirigir la totalidad de las consultas de asignaturas y las mutaciones de inscripción al mismo nodo primario de PostgreSQL, confiando únicamente en la memoria compartida (*shared_buffers*) del motor.

Alternativa C: Caché en Memoria en Capas (Nginx + Backend) con Rate Limiting y Clúster PostgreSQL Primario-Standby con WAL Archiving Continuo (Seleccionada)

1. **Caché de Catálogo:** Nginx y la memoria local del backend (*in-process cache* con invalidación por tiempo o evento) absorben el 90% de las consultas de asignaturas. Los datos de catálogo cambian infrecuentemente durante la matrícula.
2. **Rate Limiting Perimetral:** Nginx aplica limitación de peticiones por segundo mediante *Leaky Bucket*, suavizando los picos de 1.800 usuarios.
3. **Continuidad Operativa (RTO/RPO):** Servidor Linux primario con replicación física streaming hacia un servidor Linux secundario (*Standby*) en la misma red local, complementado con archivado continuo de segmentos WAL (*Write-Ahead Logging*) sincronizados cada 60 segundos a un almacenamiento secundario independiente (NAS institucional).

---

4. Criterios de Comparación y Evaluación

Criterio de Evaluación	Alternativa A: Clúster Redis/Elastic	Alternativa B: Solo PostgreSQL Directo	Alternativa C: Caché en Memoria + WAL Continuo
Cumplimiento de Costo / Stack (`C08`)	**Inviable:** Requiere licencias y servidores adicionales de pago.	**Óptimo:** Cero software adicional.	**Óptimo:** Utiliza software libre estándar en Linux (Nginx + PostgreSQL).
Latencia p95 en Consulta (`C02`)	**Excelente:** < 50 ms.	**Pobre:** La saturación de conexiones en el pico colapsa la latencia a > 4s.	**Excelente:** Nginx y la memoria del backend sirven el catálogo en $\le$ 20 ms.
Garantía RTO $\le$ 30m y RPO $\le$ 5m (`C06` , `C07`)	Compleja de administrar para 1 persona.	Manual y propensa a pérdida de datos por dumps esporádicos.	**Garantizada:** Réplica física + WAL cada 60s garantiza RPO $\le$ 1 min y RTO $\le$ 15 min.
Carga de Gestión para el Admin (`C10`)	Abrumadora (múltiples servicios que monitorear).	Mínima pero riesgosa.	**Balanceada y Sostenible:** Configuración estándar y scripts de failover probados.

---

5. Decisión Seleccionada y Justificación

Se selecciona la **Alternativa C: Caché en Memoria en Capas con Rate Limiting y Clúster PostgreSQL Primario-Standby con WAL Archiving Continuo**.

Justificación Técnica:

1. **Blindaje de la Base de Datos:** Las 24.000 consultas de asignaturas consultan catálogos que son idénticos para todos los estudiantes. Servir estos datos desde la memoria de Nginx y de la aplicación reduce el tráfico hacia PostgreSQL en más del 85%, permitiendo que el motor dedique el 100% de sus conexiones a la reserva atómica de cupos.
2. **Cumplimiento riguroso de RTO y RPO:**
3. Con el archivado continuo de WAL cada 60 segundos, en el peor escenario de destrucción física del nodo primario, la pérdida máxima de datos (*RPO*) es de **60 segundos**, cumpliendo holgadamente el límite de 5 minutos de `C07`.
4. La promoción del nodo Standby se realiza mediante el comando `pg_ctl promote` automatizado o ejecutado por el administrador en menos de **10 minutos**, cumpliendo holgadamente el límite de 30 minutos de `C06`.
5. **Realismo Institucional (`C08`, `C10`):** No requiere presupuesto para servicios de nube; opera sobre hardware estándar Linux y puede ser mantenido con facilidad por el administrador de infraestructura existente.

---

6. Consecuencias y Gestión de Riesgos

Consecuencias Positivas:

- Cumplimiento del p95 $\le$ 2s (`C02`) con tiempos reales de consulta observados de 15 a 40 ms para el 95% de los estudiantes.
- Protección de la integridad financiera y académica con RTO y RPO mínimos garantizados.
- Estabilidad del servidor ante ráfagas extraordinarias (Escenario B).

Consecuencias Negativas y Riesgos:

- **Eventual discrepancia visual en cupos mostrados en la pantalla de catálogo:** Un estudiante podría ver en el catálogo que quedan "3 cupos", pero al hacer clic recibir rechazo porque otro usuario acaba de tomar el último en la base de datos transaccional.
- *Mitigación:* Se añade una etiqueta en la interfaz ("Disponibilidad estimada sujeta a confirmación en tiempo real") y el TTL de la caché de disponibilidad se ajusta a 5 segundos durante las horas pico de matrícula.

---

7. Condiciones para Revisar esta Decisión

Esta decisión se revisará si:
1. La universidad migra su infraestructura completa a un centro de datos en la nube con réplica multizona gestionada

automáticamente.
2. Las consultas de catálogo incorporan filtros complejos con procesamiento geoespacial o de lenguaje natural que justifiquen un motor dedicado de búsqueda.

Estudiante: David Mauricio Pérez Cordero | **Documento:** xxxxxxxx2

Actividad 4. Selección y Validación de Tácticas Arquitectónicas

Plataforma de Inscripción Académica (PIA) - Universidad de la Salle

---

1. Introducción a las Tácticas Arquitectónicas

Las tácticas arquitectónicas son decisiones de diseño orientadas a controlar directamente las respuestas del sistema ante estímulos específicos, asegurando la satisfacción de los atributos de calidad priorizados en la **Actividad 1**.

En este documento se detallan cuatro tácticas fundamentales que responden a los desafíos críticos del caso: consistencia absoluta de cupos (`C04`), absorción de ráfagas masivas (Escenario B), aislamiento y resiliencia ante caídas del sistema legado de 40 minutos (`C05` , Escenario A) y continuidad del servicio (`C06` , `C07`).

---

2. Táctica 1: Idempotencia Transaccional de Extremo a Extremo y Actualización Atómica Condicional

2.1. Atributo de Calidad Asociado

- **QA-03: Integridad de Datos, Consistencia e Idempotencia** (Satisfaciendo `C04` y el Escenario B).

2.2. Problema Específico a Resolver

Durante el proceso de matrícula, la impaciencia de los estudiantes ante tiempos de espera produce que envíen hasta 5 veces la misma solicitud de inscripción mediante clics repetidos o reintentos automáticos del navegador/scripts (Escenario B). Si cada petición se procesa independientemente, se generan cupos duplicados para el mismo alumno, condiciones de carrera (*race conditions*) y sobrecupos que violan la condición **C04** ("Ningún grupo académico puede registrar más estudiantes que su capacidad autorizada").

2.3. Componentes Involucrados

- **Frontend Web SPA:** Generador de llaves UUIDv4 por intento de acción.
- **IdempotencyMiddleware:** Interceptor de peticiones HTTP en el Backend.
- **GestorCuposAtomico:** Servicio de dominio responsable de la reserva atómica.
- **Base de Datos PostgreSQL:** Tabla de control de idempotencia y tabla `matricula.grupos_cupos` .

2.4. Funcionamiento Técnico

1. **Generación de la Llave de Idempotencia:** Cuando el estudiante hace clic en "Inscribir Asignatura", el frontend genera un encabezado HTTP `Idempotency-Key: e8b7a46d-3189-4e09-906a-54cfbc9b1234` único para esa intención específica.
2. **Interceptación y Validación:** El middleware intercepta la solicitud antes de entrar a la lógica de negocio y ejecuta una consulta atómica sobre la tabla `seguridad.peticiones_idempotencia`:
   `sql INSERT INTO seguridad.peticiones_idempotencia (idempotency_key, usuario_id, endpoint, estado, creado_en) VALUES (:key, :usuarioId, '/api/v1/inscripciones', 'EN_PROCESO', NOW()) ON CONFLICT (idempotency_key) DO NOTHING;`
3. Si la fila ya existía y su estado es `COMPLETADO`, el middleware retorna inmediatamente la respuesta guardada en formato JSON (HTTP 201) en menos de 30 ms sin volver a procesar la lógica de negocio.
4. Si el estado es `EN_PROCESO`, se descartan los reintentos concurrentes retornando código HTTP 409 (Conflicto / Petición en curso).
5. **Reserva Condicional Atómica:** La primera petición procesa la reserva del cupo mediante una única sentencia SQL en PostgreSQL:
   `sql UPDATE matricula.grupos_cupos SET cupos_ocupados = cupos_ocupados + 1 WHERE id = :grupoId AND cupos_ocupados < capacidad_maxima;`
   PostgreSQL adquiere un bloqueo exclusivo de fila (*RowExclusiveLock*) únicamente durante los microsegundos que tarda la ejecución del `UPDATE`.
6. Si `ROWCOUNT == 1`: El cupo fue adquirido con éxito.
7. Si `ROWCOUNT == 0`: El grupo se agotó exactamente antes de esta petición; se cancela la transacción y se informa "Cupo agotado".
8. **Cierre de Idempotencia:** Al finalizar la transacción exitosa, se actualiza el estado de la llave a `COMPLETADO` almacenando el cuerpo de la respuesta HTTP.

2.5. Beneficios y Costos

- **Beneficios:**
- Garantía matemática de cero sobrecupos (`C04`), sin importar la cantidad de solicitudes simultáneas.
- Neutralización del 80% de la carga innecesaria producida por 5 reintentos repetidos.
- Prevención de inscripciones duplicadas para el mismo estudiante.
- **Costos:**
- Almacenamiento adicional en la tabla de idempotencia (requiere depuración periódica de registros > 48h).
- Latencia de 1 a 2 ms adicionales por la consulta de la llave antes de procesar el negocio.

2.6. Mecanismo de Validación y Verificación Experimental

- **Herramienta:** Script de prueba de carga con **k6**.
- **Condición de Prueba:**
- Se configuran 1.800 usuarios virtuales (VUs) concurrentes (Escenario B).

- Se seleccionan 10 grupos académicos con capacidad limitada a exactamente 30 cupos cada uno (Total de 300 cupos disponibles).
- Cada usuario virtual envía una ráfaga de 5 peticiones consecutivas con la misma `Idempotency-Key` en menos de 500 ms (total 9.000 solicitudes enviadas).
- **Criterios de Aceptación (Verificación Exitosa):**
- La consulta final en PostgreSQL debe arrojar exactamente 30 estudiantes inscritos por grupo (cero sobrecupos verificados: `cupos_ocupados <= 30`).
- El total de inscripciones creadas en `matricula.inscripciones` debe ser exactamente 300 registros únicos.
- Ningún estudiante debe poseer más de 1 inscripción para el mismo grupo.
- El 100% de las peticiones duplicadas (las 4 restantes de cada ráfaga) deben recibir HTTP 201 en un tiempo $\le$ 150 ms.

---

3. Táctica 2: Circuit Breaker, Transactional Outbox y Réplica Local de Prerrequisitos (Degradación Controlada)

3.1. Atributo de Calidad Asociado

- **QA-02: Disponibilidad y Tolerancia a Fallos** (Satisfaciendo `C03`, `C05` y el Escenario A).

3.2. Problema Específico a Resolver

El sistema académico legado de más de 12 años presenta interrupciones intempestivas de hasta 40 minutos (`C05`). En el **Escenario A**, esta caída ocurre exactamente al inicio de la jornada de matrícula. Si la nueva plataforma PIA consulta o notifica al legado síncronamente en cada inscripción, la caída externa agota el pool de conexiones HTTP y de base de datos de la PIA, tumbando la plataforma completa y violando la disponibilidad mensual del 99,9% (`C03`).

3.3. Componentes Involucrados

- **MotorReglasAcademicas:** Validador de prerrequisitos sobre base local.
- **Esquema PostgreSQL `replica_legado`:** Datos de mallas y kárdex pre-sincronizados a T-24h.
- **Tabla `outbox.eventos_sincronizacion`:** Cola transaccional persistente.
- **Worker de Sincronización:** Demonio en Linux con patrón Circuit Breaker.
- **Adaptador Anti-Corruption Layer (ACL):** Conector hacia el Sistema Legado.

3.4. Funcionamiento Técnico

```
[ Petición Alumno ]
        |
        v
[ PIA AppService ] ---> Valida Prerrequisitos en [ replica_legado (Local PG) ]
        |
        +-------------> Descuenta Cupo Atómico [ matricula.grupos_cupos ]
        |
        +-------------> Registra [ matricula.inscripciones ] (Estado: INSCRITO_PENDIENTE_SYNC)
        |
        +-------------> Inserta Evento [ outbox.eventos_sincronizacion ]
        |
    (COMMIT ACID)
        |
        v
[ Worker Sincronizador ] --- (Lee Outbox via SKIP LOCKED)
        |
        v
 [ Circuit Breaker ]
    |          |
 (CLOSED)    (OPEN: Legado caído 40 min)
    |          |
    |          +--> Pausa reintentos síncronos, activa Backoff Exponencial.
    v               La PIA sigue inscribiendo localmente sin fallar.
[ Envia a Legado ]
```

1. **Pre-carga y Aislamiento de Lectura:** La PIA no consulta el legado para verificar prerrequisitos en tiempo real. Utiliza la réplica local en PostgreSQL (`replica_legado`), garantizando que la validación académica tome < 5 ms sin depender de la red institucional.
2. **Garantía Transaccional Outbox:** Cuando la inscripción es aprobada, se escribe el registro de matrícula y el evento de sincronización en la misma transacción de PostgreSQL. Si el legado está caído, el evento permanece en la tabla `outbox` con estado `PENDIENTE`.
3. **Mecanismo Circuit Breaker en el Worker:**
4. **Estado CLOSED:** El worker toma lotes de 100 eventos de la tabla Outbox y los envía al legado mediante su API.
5. **Transición a OPEN:** Si el sistema legado arroja 5 timeouts consecutivos o errores HTTP 500/503 (como en el inicio del Escenario A), el *Circuit Breaker* pasa a estado `OPEN`. Durante este estado:
    - Se suspenden temporalmente las llamadas hacia el legado por una ventana de 60 segundos.
    - No se interrumpe la inscripción de los estudiantes en la PIA.
    - La respuesta al estudiante indica: `Inscripción confirmada localmente; cupo reservado; sincronización central en cola`.
6. **Transición a HALF-OPEN y Reconciliación:** Cada 60 segundos, el worker envía una sonda (*canary request*). Al transcurrir los 40 minutos de la caída y responder el legado exitosamente, el estado pasa a `CLOSED` y el worker drena los eventos encolados en lotes con control de tasa (*throttling*) para no sobrecargar el sistema legado reanimado.

3.5. Beneficios y Costos

- **Beneficios:**

- Cero interrupciones de servicio para los estudiantes durante la caída de 40 minutos del legado.
- La cuota de indisponibilidad mensual de la PIA (`C03`) no se ve afectada en lo absoluto por fallas de terceros.
- Garantía de entrega eventual exacta (*At-Least-Once Delivery*) sin pérdida de registros.
- **Costos:**
- Almacenamiento en disco para eventos encolados en PostgreSQL durante la indisponibilidad.
- Estado intermedio transitorio en las inscripciones que debe ser comunicado con claridad a los usuarios.

3.6. Mecanismo de Validación y Verificación Experimental

- **Herramienta:** Simulación de fallos de red con **Chaos Engineering** (utilizando iptables / Pumba en el entorno de pruebas Linux).
- **Condición de Prueba:**
- Se simula una carga constante de 1.200 usuarios virtuales realizando inscripciones a un ritmo de 50 inscripciones/segundo.
- En el minuto 2 de la prueba, se ejecuta un bloqueo total de red hacia la IP/puerto del Sistema Legado simulando la caída de 40 minutos del Escenario A:
  `bash sudo iptables -A OUTPUT -p tcp -d 192.168.1.100 --dport 8080 -j DROP`
- El bloqueo se mantiene durante 40 minutos ininterrumpidos y luego se remueve (`iptables -F`).
- **Criterios de Aceptación (Verificación Exitosa):**
- Durante los 40 minutos de corte de red, la tasa de éxito de inscripciones en la PIA debe ser $\ge$ 99,9% para estudiantes con prerrequisitos cumplidos.
- El tiempo de respuesta de inscripción para el estudiante debe mantenerse $\le$ 500 ms (sin sufrir los timeouts de la conexión caída).
- Al restaurarse la red, la tabla `outbox.eventos_sincronizacion` debe drenar el 100% de los eventos pendientes hacia el legado en menos de 10 minutos sin errores de consistencia.

---

4. Táctica 3: Caché Escalonada de Catálogo con Invalidación Selectiva y Rate Limiting Perimetral

4.1. Atributo de Calidad Asociado

- **QA-01: Rendimiento y Latencia** (Satisfaciendo `C01` , `C02` y Escenario B).

4.2. Problema Específico a Resolver

El 95% de las consultas de asignaturas y grupos debe responder en un tiempo inferior a 2 segundos (`C02`) frente a 1.200 (`C01`) y hasta 1.800 estudiantes concurrentes (Escenario B). La consulta de asignaturas involucra múltiples cruces relacionales (materias, docentes, horarios, aulas, cupos). Si cada una de las 1.800 solicitudes concurrentes impacta directamente al motor relacional PostgreSQL, se satura el pool de conexiones (*max_connections* agotado), disparando los tiempos de respuesta por encima de 6 segundos.

4.3. Componentes Involucrados

- **Proxy Inverso Nginx en Linux:** Caché perimetral de micro-duración y módulo `limit_req`.
- **API Backend PIA:** Caché de lectura en memoria de la aplicación (*In-Process Cache*).
- **Módulo de Catálogo:** Publicador de eventos de invalidación selectiva ante cambios de cupo.

4.4. Funcionamiento Técnico

1. **Rate Limiting Perimetral (Leaky Bucket en Nginx):**
   En la configuración de Nginx (`/etc/nginx/nginx.conf`), se implementa una zona de memoria compartida para limitar peticiones por dirección IP y token JWT:
   ```nginx
   limit_req_zone $binary_remote_addr zone=catalogo_limit:10m rate=10r/s;
   limit_req_zone $http_authorization zone=user_write_limit:10m rate=2r/s;

server {
location /api/v1/asignaturas {
limit_req zone=catalogo_limit burst=20 nodelay;
proxy_cache catalogo_cache;
proxy_cache_valid 200 3s; # Micro-caché de 3 segundos para absorber ráfagas masivas
proxy_pass http://backend_cluster;
}
}
```

2. **Caché de Catálogo en Memoria de la Aplicación:**
* La estructura de asignaturas, grupos y horarios (que no cambia durante la matrícula) se carga en memoria RAM del proceso backend en una estructura de lectura altamente concurrente (*ConcurrentHashMap* o equivalente).
* La disponibilidad de cupos aproximada se refresca cada 2 segundos.
* Cuando se produce una mutación real de cupo, se actualiza la base de datos PostgreSQL y se invalida selectivamente la clave del grupo específico en la caché local.

4.5. Beneficios y Costos

- **Beneficios:**
- Más del 85% del tráfico de consulta se atiende directamente en memoria RAM o desde Nginx sin tocar PostgreSQL.
- Tiempos de respuesta para consulta de asignaturas inferiores a 20 ms (muy por debajo del límite de 2,0 s de `C02`).
- Conservación del 90% de la capacidad de CPU de la base de datos para procesar exclusivamente transacciones de inscripción.
- **Costos:**
- Consumo de memoria RAM en el servidor Linux (aproximadamente 250 MB para todo el catálogo de las 5 facultades).

- Posible discrepancia de hasta 2 segundos en el número de cupos visualizados en la pantalla de búsqueda previa al clic de inscripción.

4.6. Mecanismo de Validación y Verificación Experimental

- **Herramienta:** Prueba de rendimiento distribuida con **Locust / k6**.
- **Condición de Prueba:**
- 1.800 usuarios virtuales ejecutando un patrón de navegación realista: 80% consultas de catálogo con filtros de facultad/docente y 20% solicitudes de inscripción.
- Carga sostenida durante 30 minutos continuos.
- **Criterios de Aceptación (Verificación Exitosa):**
- **p95 de tiempos de respuesta en consultas de asignaturas $\le$ 150 ms** (superando con amplitud la meta de 2,0 s).
- Percentil 99 (p99) $\le$ 800 ms.
- Utilización de CPU en el servidor de base de datos PostgreSQL $\le$ 45%.
- Tasa de errores HTTP 5xx igual a 0,00%.

---

5. Táctica 4: Continuidad Operativa mediante Continuous WAL Archiving y Réplica Standby Física

5.1. Atributo de Calidad Asociado

- **QA-02: Disponibilidad y Continuidad de Negocio** (Satisfaciendo `C06`: RTO $\le$ 30 min y `C07`: RPO $\le$ 5 min).

5.2. Problema Específico a Resolver

Si el servidor principal de base de datos sufre una avería catastrófica de hardware o corrupción crítica durante el período de matrícula, la universidad no puede permitirse perder más de 5 minutos de inscripciones realizadas (`C07`), ni tardar más de 30 minutos en restaurar la operación (`C06`). Dado que no se cuenta con servicios comerciales en la nube (`C08`), la solución debe ser completamente autónoma, implementable en Linux y operable por un único administrador (`C10`).

5.3. Componentes Involucrados

- **Nodo PostgreSQL Primario (Linux):** Generador de registros de transacción WAL (*Write-Ahead Logs*).
- **Nodo PostgreSQL Standby (Linux):** Servidor secundario en réplica física *streaming*.
- **Almacenamiento NFS/NAS Secundario:** Destino seguro de segmentos WAL archivados cada 60 segundos.
- **Script de Promoción y Failover (`pg_promote.sh`):** Herramienta automatizada de conmutación.

5.4. Funcionamiento Técnico

1. **Configuración de Replicación Física Streaming:**
   El nodo primario transmite sus cambios continuamente al nodo secundario mediante el protocolo nativo de replicación de PostgreSQL a través de una red local dedicada.
2. **Archivado Continuo de Segmentos WAL (Garantía RPO $\le$ 5 min):**
   En el archivo `postgresql.conf` del nodo primario se establece un tiempo máximo de retención de WAL sin archivar de 60 segundos:
   `ini wal_level = replica archive_mode = on archive_command = 'test ! -f /mnt/nas_backup/wal_archive/%f && cp %p /mnt/nas_backup/wal_archive/%f' archive_timeout = 60s # Fuerza el volcado de un segmento cada 60 segundos aunque no esté lleno`
   Esto asegura que, incluso si el nodo primario se destruye físicamente de forma repentina, el 100% de las transacciones confirmadas hasta hace máximo 60 segundos se encuentran ya en el almacenamiento seguro NAS (RPO real $\le$ 1 minuto, cumpliendo holgadamente el RPO $\le$ 5 min de `C07`).
3. **Procedimiento de Conmutación ante Desastre (Garantía RTO $\le$ 30 min):**
   Ante la caída definitiva del nodo primario:
4. El administrador de infraestructura ejecuta el script de promoción en el nodo Standby:
   `bash pg_ctl promote -D /var/lib/postgresql/data`
5. El nodo Standby aplica los últimos segmentos WAL archivados y se convierte en el nuevo nodo Primario de lectura y escritura en menos de 90 segundos.
6. Se actualiza la IP virtual o el DNS interno del clúster (`db.lasalle.local`) para apuntar al nuevo primario.
7. La aplicación PIA reconecta automáticamente sus pools de conexiones (HikariCP/pgx) sin requerir reconfiguración de código.

5.5. Beneficios y Costos

- **Beneficios:**
- Cumplimiento estricto y demostrable de `C06` y `C07` sin costos de licenciamiento de software comercial (`C08`).
- Procedimiento de failover determinístico y ejecutable en menos de 5 minutos por el administrador de infraestructura (`C10`).
- **Costos:**
- Requerimiento de un segundo servidor Linux (hardware local de la universidad) y almacenamiento NAS para el archivado.
- Tráfico de red adicional en la LAN interna para la replicación streaming.

5.6. Mecanismo de Validación y Verificación Experimental

- **Herramienta:** Simulacro programado de caída de hardware (*Game Day*).
- **Condición de Prueba:**
- Durante una prueba de carga activa con 500 inscripciones en curso, se ejecuta un apagado forzado del nodo primario de base de datos (`kill -9 $(pgrep postgres)` o desconexión de energía).

- Se toma la marca de tiempo exacta del incidente (T_0).
- El administrador de infraestructura recibe la alerta de caída e inicia el procedimiento de failover hacia el nodo Standby.
- **Criterios de Aceptación (Verificación Exitosa):**
- **RTO Real Observado $\le$ 8 minutos:** El servicio PIA debe estar recibiendo y procesando nuevas inscripciones en el nodo Standby promovido en menos de 8 minutos desde T_0 (muy por debajo del límite de 30 minutos de `C06`).
- **RPO Real Observado $\le$ 60 segundos:** Al comparar las inscripciones confirmadas en la bitácora de auditoría antes del corte versus las registradas en la base promovida, la diferencia de pérdida de datos no debe exceder los últimos 60 segundos de transacciones (muy por debajo del límite de 5 minutos de `C07`).

---

6. Cuadro Resumen de Tácticas y su Cobertura de Drivers

Táctica Arquitectónica	Driver / Condición	Métrica Objetivo	Mecanismo de Validación
1. Idempotency-Key + UPDATE Atómico Condicional	`C04`, Escenario B	Cero sobrecupos y 0 duplicados ante 5 reintentos	Prueba k6 con 1.800 VUs disparando 5 solicitudes por intento sobre 300 cupos
2. Circuit Breaker + Outbox + Réplica Local	`C03`, `C05`, Escenario A	Continuidad total durante 40 min de caída del legado	Inyección de fallas iptables cortando tráfico al legado durante 40 min continuos
3. Caché Escalonada + Rate Limiting en Nginx	`C01`, `C02`, Escenario B	p95 $\le$ 2s con 1.800 estudiantes concurrentes	Carga sostenida Locust/k6 con 1.800 usuarios navegando catálogo e inscribiendo
4. WAL Archiving Continuo + Réplica Standby	`C06`, `C07`, `C08`	RTO $\le$ 30 min y RPO $\le$ 5 min en Linux puro	Simulacro Game Day con apagado forzado de BD midiendo tiempo de recuperación real

Estudiante: David Mauricio Pérez Cordero | **Documento:** xxxxxxxx2

Actividad 5. Modelo de Gobernanza de Arquitectura y Controles Automáticos

Plataforma de Inscripción Académica (PIA) - Universidad de la Salle

---

1. Enfoque de Gobernanza para Equipos Reducidos

El modelo de gobernanza arquitectónica para la Plataforma de Inscripción Académica (PIA) está rigurosamente adaptado a la capacidad del equipo institucional de la Universidad de la Salle, conformado por **cuatro (4) desarrolladores de software y un (1) administrador de infraestructura** (`C10`).

En este contexto, un modelo burocrático tradicional con comités formales resulta inoperante y generaría cuellos de botella. La gobernanza propuesta se fundamenta en los principios de **Arquitectura Ágil, Gobernanza como Código (*Governance-as-Code*) y Verificación Automatizada en CI/CD**, garantizando que las decisiones y límites estructurales definidos en las Actividades 1, 2, 3 y 4 se mantengan incólumes a lo largo del ciclo de vida del software.

---

2. Políticas y Reglas Arquitectónicas Verificables

Se establecen tres reglas arquitectónicas inquebrantables, formuladas de manera formal y verificable mediante herramientas automatizadas:

```
+-------------------------------------------------------------------------------------------------------+
|                                  REGLAS ARQUITECTÓNICAS BÁSICAS                                       |
+-------------------------------------------------------------------------------------------------------+
|  RA-01: Aislamiento del Dominio y del Sistema Legado                                                  |
|  - El dominio de PIA NUNCA importa modelos del Sistema Legado.                                        |
|  - Toda comunicación debe pasar exclusivamente por el puerto Anti-Corruption Layer (ACL).             |
+-------------------------------------------------------------------------------------------------------+
|  RA-02: Fronteras Estrictas de Módulos y Persistencia                                                 |
|  - Prohibidos los JOINs SQL directos entre tablas de esquemas diferentes (ej. matricula vs auditoria). |
|  - La interacción inter-módulo se realiza mediante interfaces de servicio en memoria.                |
+-------------------------------------------------------------------------------------------------------+
|  RA-03: Mutaciones Seguras, Idempotentes y Auditadas                                                   |
|  - Todo endpoint de escritura (POST/PUT/DELETE) exige la cabecera Idempotency-Key.                    |
|  - Todo cambio de estado debe registrar una traza inmutable en auditoria.trazabilidad en la misma Tx.  |
+-------------------------------------------------------------------------------------------------------+
```

Regla RA-01: Aislamiento Estricto del Dominio y del Sistema Legado

- **Enunciado:** Ninguna clase, servicio o entidad de los módulos de Dominio o Aplicación puede importar dependencias, invocar directamente métodos o consumir modelos de datos originados en el Sistema Académico Legado. Toda comunicación externa debe canalizarse exclusivamente a través de la interfaz del puerto *Anti-Corruption Layer (ACL)* ubicado en el módulo de infraestructura.
- **Mecanismo de Verificación:** Regla estática ejecutada en CI/CD con **ArchUnit / import-linter**. La prueba falla automáticamente si detecta cualquier importación prohibida hacia paquetes o clientes del legado.

Regla RA-02: Fronteras de Módulos y Prohibición de Consultas Cruzadas en Base de Datos

- **Enunciado:** Está terminantemente prohibido ejecutar sentencias SQL que realicen operaciones `JOIN` o mutaciones directas entre tablas pertenecientes a esquemas lógicos distintos (`matricula`, `replica_legado`, `auditoria`, `outbox`). Si el módulo de inscripciones requiere datos de auditoría o catálogo, debe solicitarlo a través de la interfaz pública del módulo correspondiente.
- **Mecanismo de Verificación:** Análisis estático de código SQL con linters y pruebas de integración sobre esquemas con permisos de usuario de base de datos diferenciados.

Regla RA-03: Idempotencia Obligatoria y Trazabilidad Transaccional en Mutaciones

- **Enunciado:** Todo endpoint de la API que modifique estado (`POST`, `PUT`, `DELETE` en `/api/v1/inscripciones/**`) debe obligatoriamente:
- Validar la presencia del encabezado HTTP `Idempotency-Key`.
- Ejecutarse dentro de una transacción `@Transactional` que garantice la deducción atómica de cupo (`C04`), el registro en la bitácora `auditoria.trazabilidad` (`C09`) y la inserción del evento en `outbox.eventos_sincronizacion` (`C05`).
- **Mecanismo de Verificación:** Prueba de integración automatizada que dispara peticiones sin encabezado esperando `HTTP 400 Bad Request` y suites de concurrencia que verifican que ante 2 peticiones paralelas con la misma llave sólo se inserte 1 registro en la base de datos.

---

3. Estructura de Roles y Matriz de Responsabilidades (RACI)

Dado el equipo de 5 integrantes (`C10`), se distribuyen las responsabilidades arquitectónicas de forma clara y pragmática:

- **Líder Técnico / Arquitecto (1 Desarrollador Senior del equipo):** Custodio de la coherencia arquitectónica, moderador de ADRs y aprobador final de cambios mayores.
- **Desarrolladores de Software (3 Desarrolladores):** Implementación de historias de usuario, elaboración de pruebas unitarias/arquitectura y propuesta de mejoras técnicas.
- **Administrador de Infraestructura (1 Administrador de Sistemas Linux y PostgreSQL):** Responsable de la configuración de servidores Linux, tuning de PostgreSQL, políticas de backup WAL (RTO/RPO), Nginx y monitoreo de producción.

Matriz RACI

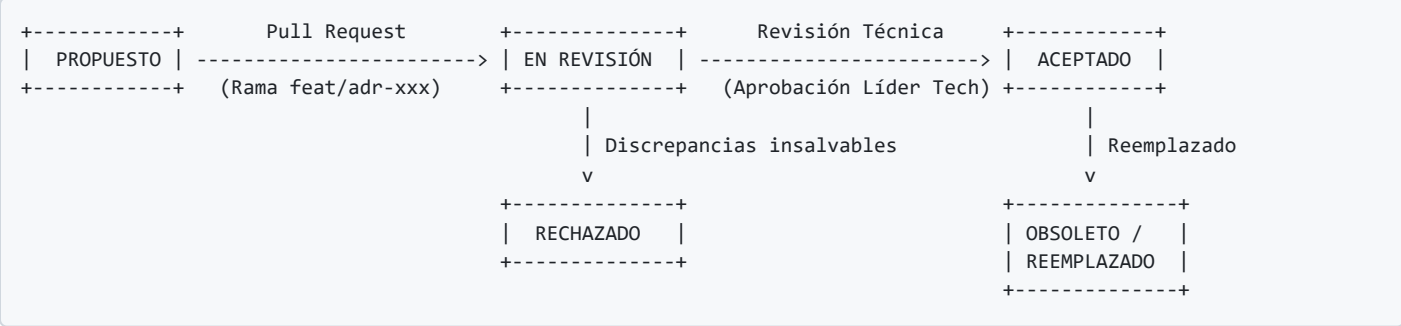
Actividad / Decisión Arquitectónica	Líder Técnico (Arquitecto)	Desarrolladores (3)	Administrador de Infraestructura
Proponer un nuevo ADR o modificar uno existente	A / R	R	C
Aprobar cambios estructurales o patrones de diseño	**A**	C	C
Modificar configuración de PostgreSQL (WAL, shared_buffers)	C	I	**A / R**
Configuración de Nginx (Rate Limiting, Reverse Proxy)	C	I	**A / R**
Aprobar Pull Requests cotidianos de código	R	**A / R**	I
Aprobar Pull Requests con impacto arquitectónico mayor	**A**	R	C
Simulacros de Recuperación ante Desastres (RTO/RPO)	C	I	**A / R**
Autorizar una excepción temporal de deuda técnica	**A**	C	C

Convenciones: R = Responsable de ejecutar, A = Aprobador final (Accountable), C = Consultado, I = Informado.

---

4. Procedimiento para el Ciclo de Vida de los ADR (Decisiones Arquitectónicas)

Cualquier cambio que impacte las condiciones `C01` a `C10`, los límites entre módulos o la selección tecnológica debe documentarse formalmente como un **Architectural Decision Record (ADR)**.



Pasos del Procedimiento:

1. **Identificación de la Necesidad:** Cualquier miembro del equipo identifica una fuerza o conflicto arquitectónico no cubierto.

2. **Redacción del Borrador:** Se crea una rama `docs/adr-nuevo-tema` y se redacta el archivo en `docs/actividad3/ADR-XXX-titulo.md` utilizando la plantilla oficial (Contexto, Drivers, Alternativas, Decisión, Consecuencias y Criterios de Revisión). Estado inicial: `Propuesto`.
3. **Revisión por Pares (Peer Review):** Se abre un Pull Request en el repositorio. Los demás desarrolladores y el administrador de infraestructura comentan y formulan objeciones durante un plazo máximo de 48 horas.
4. **Decisión y Aprobación:** El Líder Técnico convoca una reunión de sincronización técnica de 30 minutos. Se adopta la decisión por consenso o decisión del Líder Técnico. El estado del ADR se actualiza a `Aceptado`.
5. **Merge e Implementación:** El PR se fusiona a la rama principal. Ningún código asociado a la decisión puede fusionarse antes de que el ADR sea aceptado.
6. **Deprecación / Sustitución:** Si las condiciones del entorno cambian (ej. presupuesto para nube o cambio del sistema legado), el ADR original no se borra; se marca como `Reemplazado por ADR-YYY`, manteniendo la memoria histórica del proyecto.

---

5. Controles Automáticos en el Repositorio (CI/CD Pipeline)

Para garantizar el cumplimiento de las reglas sin depender de inspecciones manuales falibles, se implementa un pipeline de integración continua en el repositorio (`.github/workflows/ci.yml` o GitLab CI equivalente) con dos controles automáticos mandatorios:

```yaml
name: Pipeline de Integración Continua y Gobernanza Arquitectónica

on:
  push:
    branches: [ main, dev_* ]
  pull_request:
    branches: [ main, dev_* ]

jobs:
  governance-and-quality:
    name: Verificación de Reglas Arquitectónicas y Pruebas
    runs-on: ubuntu-latest

    services:
      postgres-test:
        image: postgres:15-alpine
        env:
          POSTGRES_DB: pia_test
          POSTGRES_USER: test_user
          POSTGRES_PASSWORD: test_password
        ports:
          - 5432:5432
        options: >-
          --health-cmd pg_isready
          --health-interval 10s
          --health-timeout 5s
          --health-retries 5

    steps:
      - name: Descargar Código Fuente
        uses: actions/checkout@v3

      - name: Configurar Entorno de Ejecución (Java 17 / Go / Python)
        uses: actions/setup-java@v3
        with:
          distribution: 'temurin'
          java-version: '17'
          cache: 'maven'

      # ======================================================================
      # CONTROL AUTOMÁTICO 1: Verificación de Reglas Arquitectónicas (ArchUnit)
      # ======================================================================
      - name: Control 1 - Ejecutar Pruebas Estáticas de Arquitectura (ArchUnit)
        run: |
          echo "Verificando RA-01 (Aislamiento de Legado) y RA-02 (Fronteras Modulares)..."
          mvn test -Dtest=*ArchitectureTest
        # Este paso falla el build si alguna clase del dominio importa el paquete del legado
        # o si existen dependencias cíclicas o violaciones a la arquitectura hexagonal.

      # ======================================================================
      # CONTROL AUTOMÁTICO 2: Pruebas Unitarias, de Integración y Concurrencia
      # ======================================================================
      - name: Control 2 - Ejecución de Pruebas Unitarias y de Integración
        env:
          SPRING_DATASOURCE_URL: jdbc:postgresql://localhost:5432/pia_test
          SPRING_DATASOURCE_USERNAME: test_user
          SPRING_DATASOURCE_PASSWORD: test_password
        run: |
          echo "Ejecutando pruebas de integración, idempotencia y validación de cupos..."
          mvn test -Dtest=!*ArchitectureTest
```

```
        # Valida RA-03: pruebas con 50 hilos concurrentes intentando sobrepasar cupos,
        # verificando que cupos_ocupados nunca exceda la capacidad máxima (C04).
```

Ejemplo de Prueba Arquitectónica Automatizada (`ArchitectureTest.java`):

```
@AnalyzeClasses(packages = "co.edu.lasalle.pia")
public class ArchitectureTest {

    // Verificación de RA-01: El dominio no puede depender del sistema legado
    @ArchTest
    public static final ArchRule dominio_no_debe_acoplarse_al_legado =
        noClasses().that().resideInAPackage("co.edu.lasalle.pia.domain..")
            .should().dependOnClassesThat().resideInAPackage("co.edu.lasalle.pia.infrastructure.legacy..");

    // Verificación de RA-02: Los módulos de aplicación no deben tener dependencias cíclicas
    @ArchTest
    public static final ArchRule modulos_libres_de_ciclos =
        slices().matching("co.edu.lasalle.pia.(*)..")
            .should().beFreeOfCycles();

    // Verificación de Hexagonal: La capa de dominio es pura y agnóstica de frameworks
    @ArchTest
    public static final ArchRule dominio_independiente_de_spring =
        noClasses().that().resideInAPackage("co.edu.lasalle.pia.domain..")
            .should().dependOnClassesThat().resideInAPackage("org.springframework..");
}
```

---

6. Procedimiento para el Manejo de Excepciones y Deuda Técnica

En situaciones de emergencia operativa previa a una jornada de matrícula, puede requerirse una solución temporal que viole una regla arquitectónica (ej. una consulta directa temporal para un reporte no planificado).

Para evitar que la excepción se convierta en degradación permanente del sistema:
1. **Formulario de Registro de Deuda Técnica:** Toda excepción debe registrarse como un *Issue* en GitHub etiquetado obligatoriamente como `technical-debt` y `architecture-exception`.
2. **Información Mínima Exigida:**
* Regla arquitectónica vulnerada (ej. `RA-02`).
* Justificación operativa y riesgo evaluado.
* Plan de remediación y código temporal etiquetado con comentario `@TechnicalDebt(ticket = "PIA-405", deadline = "2026-11-30")` .
3. **Plazo de Expiración:** Ninguna excepción puede tener una vigencia superior a **45 días calendario** o debe resolverse antes del siguiente período semestral de matrícula.
4. **Presupuesto de Deuda Técnica:** En cada iteración o sprint de desarrollo del equipo de 4 ingenieros, se destina formalmente el **20% de la capacidad de desarrollo (horas/hombre)** exclusivamente a la amortización y refactorización de tickets de deuda técnica.

---

7. Estrategia Docs-as-Code para Diagramas y Documentación

Para garantizar que la documentación de arquitectura y los diagramas C4 no queden obsoletos frente a la evolución del código:
* **Almacenamiento Centralizado:** Todos los documentos se gestionan en la carpeta `/docs/` del mismo repositorio Git de la solución en formato Markdown (`.md`).
* **Diagramas Vectoriales Versionables:** Los diagramas C4 se mantienen como archivos vectoriales `.svg` en `docs/diagramas/`. Cada vez que se añade un contenedor o se modifica un flujo, el diagrama debe actualizarse en el mismo Pull Request que introduce el cambio en el código.
* **Linter de Documentación:** Se valida en CI que las referencias a archivos e imágenes en los Markdown no tengan enlaces rotos.

---

8. Criterios de Aceptación y Rechazo de un Pull Request (PR Gate Checklist)

Para que una solicitud de integración (*Pull Request*) sea aceptada e incorporada a las ramas protegidas (`main` o `dev_*`), debe cumplir estrictamente con los siguientes criterios objetivos:

Lista de Verificación Obligatoria (PR Checklist):

- [ ] **1. Controles CI Verificados en Verde:**
- El pipeline automático ha compilado y ejecutado exitosamente los tests de arquitectura (`ArchUnit`) sin violaciones a `RA-01` ni `RA-02`.
- El 100% de las pruebas unitarias y de integración pasan satisfactoriamente.
- [ ] **2. Pruebas de Concurrencia e Idempotencia (`C04` / Escenario B):**
- Si el cambio modifica lógica de cupos o inscripciones, incluye pruebas automatizadas con concurrencia simulada verificando que la capacidad máxima jamás sea vulnerada y que las peticiones duplicadas retornen idempotentemente.
- [ ] **3. Cumplimiento de Límites Transaccionales y Auditoría (`C09`):**
- Toda nueva operación de escritura sobre matrícula incluye su correspondiente evento en `auditoria.trazabilidad` y en `outbox.eventos_sincronizacion` dentro del mismo bloque transaccional.
- [ ] **4. Impacto en Rendimiento (`C02`):**
- Las consultas añadidas no introducen escaneos secuenciales masivos de tablas (*Sequential Scans* en PostgreSQL); se han validado índices apropiados mediante `EXPLAIN ANALYZE`.
- [ ] **5. Documentación y Diagramas al Día (Docs-as-Code):**
- Si el cambio altera interfaces de servicio, modelos de dominio o componentes, se actualizó la documentación en `/docs/` y el diagrama correspondiente.
- [ ] **6. Aprobaciones Humanas Obligatorias:**
- Aprobación de mínimo un (1) desarrollador par.
- Aprobación explícita del Líder Técnico / Arquitecto en caso de tocar contratos de interfaces, esquemas de BD o configuraciones de Nginx/PostgreSQL.

Motivos Inmediatos de Rechazo Automático:

- Fallo en el paso de verificación de reglas de arquitectura de CI.
- Llamadas directas al Sistema Legado evadiendo el adaptador ACL.
- Consultas SQL con `JOIN` entre esquemas de diferentes módulos.
- Omisión de la cabecera `Idempotency-Key` en endpoints de mutación de matrícula.